

Embedding Compression for Dense Retrieval: A Fair Empirical Comparison of Post-Hoc and Training-Time Methods

Sajil Awale

A THESIS

**Submitted in partial fulfillment of the requirements
for the degree of Master of Science**

in

The Department of Computer Science

to

The Graduate School

of

The University of Alabama in Huntsville

August 2026

Approved by:

Dr. [Research Advisor Name], Research Advisor

Dr. [Committee Chair Name], Committee Chair

Dr. [1st Committee Member Name], Committee Member

Dr. [2nd Committee Member Name], Committee Member

Dr. [3rd Committee Member Name], Committee Member

Dr. [Department Chair Name], Department Chair

Dr. [College Dean Name], College Dean

Dr. Jon Hakkila, Graduate Dean

Abstract

Embedding Compression for Dense Retrieval: A Fair Empirical Comparison of Post-Hoc and Training-Time Methods

Sajil Awale

A thesis submitted in partial fulfillment of the requirements
for the degree of Master of Science

Computer Science

The University of Alabama in Huntsville

August 2026

Dense vector embeddings are central to modern retrieval-augmented generation (RAG) pipelines and large-scale search systems, but their memory footprint becomes a critical bottleneck at scale. This thesis presents a controlled empirical comparison of post-hoc and training-time embedding compression methods, evaluated under identical training data and benchmark conditions across three encoder architectures: BGE-base-en-v1.5, RoBERTa-base, and MPNet-base. Seven compression methods are studied: INT8 scalar quantization, binary quantization, Product Quantization, TurboQuant, Straight-Through Estimator (STE) binarization, Annealed Tanh binarization, and Matryoshka Representation Learning (MRL), including systematic stacked combinations that pair MRL dimensionality reduction with precision-reducing methods. **[TODO: 2–3 sentences of key results, e.g., main finding on training-time vs. post-hoc; which binarization strategy wins; MRL stacking outcome; cross-architecture consistency.]** Results are presented as Pareto curves on NanoBEIR, offering practitioners direct guidance on method selection under a given memory budget.

Acknowledgements

I am sincerely grateful to my advisor, Dr. Tathagata Mukherjee, for his guidance, encouragement, and unwavering support throughout this research. His expertise and patience were invaluable in shaping both the direction of this work and my growth as a researcher.

I would also like to thank the members of my thesis committee, Dr. Jacob Hauenstein and Dr. Manil Maskey, for their insightful feedback and constructive critiques. Their thoughtful suggestions greatly improved the quality of this work.

I acknowledge NASA IMPACT for providing the compute resources that made the experiments in this thesis possible. I am also grateful to my colleagues from the LLM team, Nishan Pantha and Muthukumaran Ramasubramanian, for their guidance and collaborative discussions throughout this research.

Finally, I am deeply thankful to my family and friends for their constant support and encouragement. Their belief in me sustained my motivation through every challenge along the way.

Table of Contents

Abstract	ii
Acknowledgements	iv
Table of Contents	ix
List of Figures	x
List of Tables	xi
Chapter 1. Introduction	1
1.1 Motivation	1
1.2 Embedding Compression Landscape	3
1.3 Problem Statement	5
1.4 Research Questions	6
1.5 Contributions	7
1.6 Thesis Organization	8
Chapter 2. Background and Related Work	10
2.1 Dense Retrieval with Bi-Encoders	10

2.1.1	Transformer Encoders	10
2.1.2	Bi-Encoder Architecture	11
2.1.3	BGE-base-en-v1.5	13
2.1.4	RoBERTa-base	13
2.1.5	MPNet-base	14
2.2	Contrastive Learning for Dense Retrieval	15
2.2.1	The InfoNCE Objective	15
2.2.2	Multiple Negatives Ranking Loss	16
2.3	Embedding Compression Methods	16
2.3.1	Scalar Quantization	17
2.3.2	Binary Quantization	17
2.3.3	Product Quantization	18
2.3.4	TurboQuant	19
2.3.5	Straight-Through Estimator Binarization	19
2.3.6	Annealed Tanh Binarization	20
2.3.7	Matryoshka Representation Learning	21
2.4	Information Retrieval Evaluation	22
2.4.1	Ranking Metrics	22
2.4.2	Statistical Significance Testing	24

2.5	Related Work	26
Chapter 3.	Datasets	30
3.1	Training Data	30
3.1.1	Dataset Sources	31
3.1.2	Weighted Batch Sampling	33
3.2	Evaluation Benchmark	34
3.2.1	NanoBEIR	34
Chapter 4.	Methodology	37
4.1	Experimental Design	37
4.2	Training Infrastructure and Hyperparameters	39
4.2.1	Backbone Models	39
4.2.2	Training Setup	39
4.2.3	Hyperparameters	39
4.3	Baseline Fine-Tuning	41
4.4	Training-Time Compression Experiments	41
4.4.1	STE Binarization-Aware Training	42
4.4.2	MRL Fine-Tuning	42
4.4.3	Joint MRL and STE Binarization-Aware Training	43

4.4.4	Annealed Tanh Binarization-Aware Training	43
4.4.5	Joint MRL and Annealed Tanh Binarization-Aware Training	44
4.5	Post-Hoc Compression	44
4.5.1	INT8 Scalar Quantization	44
4.5.2	Binary Quantization	44
4.5.3	Product Quantization	45
4.5.4	TurboQuant	45
4.6	Stacked Combinations	45
4.7	Evaluation Framework	46
4.7.1	Embedding Cache and Transform Pipeline	46
4.7.2	Metrics	47
4.7.3	Memory Budget and Compression Ratio	47
4.7.4	Statistical Significance Testing	47
	Experiments and Results	49
5.1	Evaluation Setup	49
5.2	Float32 Baselines	51
5.3	Post-Hoc Compression	53
5.3.1	INT8 Scalar Quantisation	54

5.3.2	Binary Quantisation	54
5.3.3	Product Quantisation	55
5.3.4	TurboQuant	56
5.3.5	Cross-Method Comparison at $32\times$	57
5.4	Training-Time Compression	58
5.4.1	Binarisation-Aware Training: STE and Annealed Tanh	59
5.4.2	Matryoshka Representation Learning	61
5.5	Stacked Combinations	62
5.6	Pareto Analysis	65
5.6.1	Pareto Frontiers	65
5.6.2	Best Method at Each Compression Budget	67
5.6.3	Training-Time versus Post-Hoc Methods on the Frontier	68
5.7	Cross-Backbone Generalisability	69
5.8	Discussion	72
References		76
Appendix A: Annealing Rate Ablation		81
A.1	Effect of γ on Annealed Tanh Binarization-Aware Training . .	81

List of Figures

5.1	Mean MRR@10 vs compression ratio for TurboQuant across bit-widths and backbones.	56
5.2	Mean MRR@10 at $32\times$ compression for all post-hoc methods across backbones. Float32 fine-tuned scores shown as dashed reference lines.	57
5.3	Mean MRR@10 at $32\times$ compression for post-hoc binary, BAT, and annealed tanh ($\gamma=0.2$) across backbones.	59
5.4	Mean MRR@10 as a function of MRL truncation dimension d for all three backbones. The float32 768-dim baseline is shown as a dashed reference line for each backbone.	61
5.5	Mean MRR@10 vs compression ratio for MRL+TQ and MRL+PQ stacked combinations. Standalone TQ and PQ results are shown as dashed reference curves.	64
5.6	Quality-compression Pareto frontiers for BGE, RoBERTa, and MP-Net. Compression ratio is shown on a log scale. Filled markers indicate Pareto-optimal configurations; hollow markers are dominated.	65
5.7	Quality retention (compressed MRR@10 / float32 MRR@10) for all evaluated methods across backbones. Values above 1.0 indicate the compressed model exceeds its float32 baseline.	69

List of Tables

1.1	Float32 index size and monthly instance cost by embedding dimension	2
1.2	Organization of embedding compression methods studied in this thesis	4
3.1	Training dataset summary.	31
3.2	NanoBEIR benchmark subsets.	36
4.1	Summary of training variants and applicable post-hoc transforms.	38
4.2	Training hyperparameters.	40
5.1	NanoBEIR subsets used for evaluation.	49
5.2	Mean MRR@10 (macro-averaged over 13 NanoBEIR subsets) for pre-trained and fine-tuned float32 embeddings. Δ is the absolute change due to fine-tuning.	52
5.3	Mean MRR@10 for INT8 quantisation ($4\times$) compared with the fine-tuned float32 baseline.	54
5.4	Mean MRR@10 for post-hoc binary quantisation ($32\times$) compared with the fine-tuned float32 baseline.	55
5.5	Mean MRR@10 for PQ configurations ($32\times$) compared with the fine-tuned float32 baseline and post-hoc binary.	55
5.6	Mean MRR@10 at $32\times$ compression. Float32 fine-tuned scores are shown for reference ($r = 1\times$).	57
5.7	Mean MRR@10 at $32\times$ compression for binarisation-aware training methods, post-hoc binary, and the float32 baseline.	59
5.8	Mean MRR@10 for MRL truncations at 512 and 256 dimensions, with compression ratio and float32 baseline shown for reference.	61

5.9	Mean MRR@10 for stacked combinations, with compression ratio and memory footprint per vector. MRL float32 baselines are included for reference.	62
5.10	BGE Pareto-optimal configurations, ordered by compression ratio. Method family is indicated for each point.	66
5.11	Best-performing method and MRR@10 at four representative compression budgets for each backbone. Ties broken by compression ratio (higher preferred).	67
12	γ ablation: mean MRR@10 on NanoBEIR	81

Chapter 1. Introduction

1.1 Motivation

Dense vector retrieval has become a core dependency of modern AI systems. Every retrieval-augmented generation (RAG) pipeline [17], enterprise search engine, and question-answering service relies on the ability to find semantically relevant content in large corpora at query time. Unlike sparse methods such as TF-IDF or BM25, dense retrieval encodes queries and passages as continuous vector embeddings using bi-encoder architectures [23, 14], and measures semantic similarity by cosine or dot-product distance. This enables retrieval that generalizes well beyond exact keyword matching, which has driven rapid adoption across search, recommendation, and generative AI applications.

As retrieval corpora scale to tens of millions or billions of passages, the memory footprint of stored embeddings becomes a critical infrastructure bottleneck. A standard model such as BGE-base-en-v1.5 [34] produces 768-dimensional float32 vectors, consuming 3,072 bytes per passage. Indexing one hundred million passages therefore requires approximately 286 GB; at one billion passages this grows to approximately 2.9 TB.

Dense retrieval systems built on libraries such as FAISS [12] typically require the embedding index to reside in random-access memory (RAM) for interac-

tive query latencies: DRAM access is on the order of 100 ns, roughly three orders of magnitude faster than NVMe SSD random reads. Serving these in-memory indices requires memory-optimized cloud instances; the AWS x2gd family, for example, costs approximately \$3.80 per GB per month [1]. Table 1.1 shows how this cost scales with embedding dimension and corpus size across widely-used models.

Table 1.1: Float32 embedding index size and estimated monthly cost on memory-optimized AWS x2gd instances (\$3.80/GB/month) at two corpus scales [24, 1]. Sizes are reported in GB (SI).

Dim	Example Models	100M Passages	1B Passages
384	all-MiniLM-L6-v2, bge-small-en-v1.5	143 GB / \$543/mo	1,431 GB / \$5,435/mo
768	bge-base-en-v1.5, nomic-embed-text-v1	286 GB / \$1,087/mo	2,861 GB / \$10,871/mo
1024	bge-large-en-v1.5, mxbai-embed-large-v1	381 GB / \$1,449/mo	3,815 GB / \$14,495/mo
1536	text-embedding-3-small	572 GB / \$2,174/mo	5,722 GB / \$21,743/mo
3072	text-embedding-3-large	1,144 GB / \$4,348/mo	11,444 GB / \$43,487/mo

As Table 1.1 shows, a billion-passage index at 768 dimensions already exceeds \$10,800 per month, and the cost more than quadruples when moving to 3,072-dimensional models. These figures represent the minimum viable infrastructure: the memory requirement dictates the instance size, and therefore the total cost, including compute. The problem is growing worse as the field trends toward higher-dimensional embeddings (models such as text-embedding-3-large at 3,072 dimensions are increasingly common) and as RAG-based applications push corpus sizes into the billions of passages.

Beyond cost, larger embeddings also hurt speed: every query must read each document vector from memory, so the bigger the vectors, the fewer queries the system can serve per second. For resource-constrained environments such as on-device or edge deployments, multi-terabyte indices are simply infeasible regardless of budget. Embedding compression is therefore not merely a storage optimization; it is a prerequisite for deploying dense retrieval at scale.

Even simple compression approaches already show real promise: replacing each 32-bit float with a single bit (binary quantization) shrinks the index by $32\times$ and retains much of the retrieval quality when applied to a strong pre-trained model [24]. Compression can be pushed further by stacking multiple techniques. The open questions are how far compression can go before retrieval quality degrades substantially, and whether incorporating compression awareness into fine-tuning produces better representations than applying post-training compression. This thesis investigates both.

1.2 Embedding Compression Landscape

This thesis organizes embedding compression methods along two orthogonal axes. The first axis is *when* compression is applied: **post-hoc** methods are applied to the generated embeddings without modifying the model, while **training-time** methods incorporate compression awareness into the model’s fine-tuning objective. The second axis is *how* memory is reduced: **precision reduction** decreases the number of bits per dimension, **dimensionality reduction** decreases the number of dimensions kept, and **learning to hash** fits a codebook that maps

embeddings to discrete codes directly, rather than applying scalar quantization per dimension. Table 1.2 places the methods studied in this thesis along these two axes.

Table 1.2: Organization of embedding compression methods studied in this thesis, by when compression is applied and how memory is reduced. Abbreviations: INT8 = 8-bit integer quantization; PQ = Product Quantization; BAT = Binary-Aware Training (STE and Annealed Tanh are two training variants); STE = Straight-Through Estimator; MRL = Matryoshka Representation Learning.

	Precision Reduction	Dim. Reduction	Learning to Hash
Post-hoc	INT8, Binary, TurboQuant	—	PQ
Training-time	BAT (STE / Annealed Tanh)	MRL	—

Each method reduces embedding memory through a distinct mechanism. **8-bit integer (INT8)** quantization maps each float32 value to an 8-bit integer, yielding $4\times$ compression with minimal quality loss. **Binary quantization** maps each dimension to a single bit ($32\times$ compression), enabling fast Hamming-distance similarity search. **Product Quantization (PQ)** [11] partitions each embedding into fixed-width sub-vectors and quantizes each to a small learned codebook, achieving high compression ratios with better quality retention than scalar binary quantization. **TurboQuant** [37] applies a random rotation to each embedding before scalar quantization, distributing distortion uniformly across dimensions to approach information-theoretic compression bounds. **Binary-Aware Training (BAT)** fine-tunes the model with an inserted sign function to pro-

duce binary-friendly representations; two gradient approximation variants are studied: the Straight-Through Estimator (STE) [3], which passes gradients unchanged through the discontinuity, and Annealed Tanh, which replaces the STE with a temperature-annealed $\tanh(\beta \cdot x)$ surrogate that smoothly transitions toward the sign function as training progresses. **Matryoshka Representation Learning (MRL)** [15] trains nested subspace representations so that any leading k -dimensional prefix suffices for retrieval, enabling post-hoc truncation to any desired dimensionality at inference time.

The fundamental tension in embedding compression is between compression depth and retrieval quality. Every method in Table 1.2 trades information loss for a smaller memory footprint: reducing bits per dimension discards numerical precision, reducing dimensions discards representational capacity, and learned codes introduce quantization error. At moderate compression (INT8 at $4\times$), quality losses are negligible. At aggressive compression (binary embeddings at $32\times$, heavily truncated MRL representations, or stacked combinations), degradation becomes more substantial, and the shape of this trade-off varies significantly across methods and compression levels.

1.3 Problem Statement

The fundamental tension in embedding compression is between compression depth and retrieval quality. Every compression approach trades information for a smaller memory footprint, and the shape of this trade-off varies across methods and compression levels.

Recent work has begun to map this trade-off systematically. [?] systematically evaluate retrieval performance across multiple post-training compression techniques at various compression ratios. However, no existing study extends this comparison to compression aware fine-tuning. In most prior work [35, 8, 24], each method is evaluated on a different base model, training corpus, and benchmark. A method that appears to win may therefore reflect a better starting point rather than a genuinely better compression strategy.

A secondary gap concerns stacking. Combinations of multiple complementary compression techniques, such as dimensionality reduction combined with quantization, remain largely unstudied. Chapter 2 surveys the prior work that establishes both gaps.

This thesis addresses both gaps through a controlled empirical study. Results are reported as Pareto curves on the NanoBEIR benchmark [29], directly answering the practitioner’s question: given a memory budget, which approach should you use? The contributions are detailed in Section ??.

1.4 Research Questions

This thesis is organized around four research questions:

1. **Training-time versus post-hoc compression.** Does embedding quantization-aware training (such as BAT with STE or Annealed Tanh, or MRL) yield better retrieval quality than post-hoc compression applied to a float-trained baseline at equivalent embedding sizes?

2. **Binarization training strategy.** Between STE and Annealed Tanh Binarization, which gradient surrogate produces better binary embeddings, and how sensitive are results to the annealing rate?
3. **Compression stacking with MRL.** Does combining MRL with other compression methods (binary quantization, PQ, TurboQuant, and BAT) yield better Pareto efficiency than either technique applied independently?
4. **Cross-architecture generalizability.** Do the above findings hold consistently across different encoder architectures (BGE-base, RoBERTa-base, and MPNet-base), or are they specific to a particular model?

1.5 Contributions

This thesis makes the following contributions to the field of efficient dense text retrieval:

1. **Systematic cross-method comparison.** Post-hoc and training-time compression methods are evaluated side by side under identical training data and benchmark, across three encoder architectures (BGE-base, RoBERTa-base, MPNet-base). Both single methods and stacked combinations are covered, spanning binary, scalar, product quantization, and dimensionality reduction at multiple operating points. Prior work evaluates each method under different setups; this study provides a direct, fair comparison.
2. **Binarization strategy comparison.** STE and Annealed Tanh have each been used for retrieval binarization in prior work [3, 35], but never com-

pared directly. Both are trained under identical conditions to isolate the effect of the gradient surrogate on retrieval quality. A sweep of the temperature schedule parameter measures how sensitive Annealed Tanh is to the annealing rate.

3. **Compression stacking with MRL.** MRL is combined with five compression techniques: post-hoc binary quantization, PQ, TurboQuant, BAT (STE), and Annealed Tanh. Results establish which combinations achieve the best Pareto trade-offs and whether jointly training MRL with binarization outperforms applying the two techniques independently.
4. **Reproducible experimental pipeline.** A DDP fine-tuning setup over seven heterogeneous retrieval datasets with domain-weighted batch sampling provides the controlled training baseline. An embedding caching and transform pipeline decouples encoding from evaluation, enabling any compression variant to be benchmarked without re-encoding. Results are surfaced through an interactive Plotly dashboard with pairwise Wilcoxon signed-rank and paired t-test significance testing across all model pairs.

1.6 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 provides background on dense retrieval, contrastive learning objectives, and the embedding compression methods compared in this work. Chapter 3 describes the training and evaluation datasets, including the multi-source training corpus construction

and the NanoBEIR benchmark. Chapter 4 details the training setup and the fine-tuned float32 baseline that serves as the anchor for all downstream comparisons. Chapter 5 presents the compression methods, both post-hoc and training-time, including the derivation of the Annealed Tanh Binarization approach and its relationship to the STE-based BAT baseline. Chapter 6 reports the experimental results, including NanoBEIR evaluations and Pareto analysis across all methods. Chapter 7 concludes with a summary of findings, practical guidance on method selection at given memory budgets, and directions for future work.

Chapter 2. Background and Related Work

This chapter establishes the technical foundations on which the remainder of the thesis is built. Section 2.1 describes bi-encoder architectures for dense retrieval and introduces the three backbone models used in this thesis (BGE-base-en-v1.5, RoBERTa-base, and MPNet-base). Section 2.2 covers the contrastive learning objectives used during fine-tuning. Section 2.3 provides technical descriptions of each compression method surveyed in this thesis, including Matryoshka Representation Learning (MRL). Section 2.4 defines the retrieval evaluation metrics and significance testing procedure used throughout this thesis. Section 2.5 surveys related work on binary and quantized retrieval, differentiable quantization techniques, adaptive representations, and controlled compression evaluation, and positions this thesis relative to prior contributions.

2.1 Dense Retrieval with Bi-Encoders

2.1.1 Transformer Encoders

Modern dense retrieval systems are built on top of transformer-based language models [6]. A transformer encoder maps a token sequence of length L to a sequence of contextualized hidden states $\mathbf{H} \in \mathbb{R}^{L \times d_h}$, where d_h is the hidden dimension, through a stack of self-attention and feed-forward layers. Each self-

attention layer allows every token to attend to every other token in the sequence, producing representations that capture long-range syntactic and semantic dependencies. Bidirectional encoders such as BERT (Bidirectional Encoder Representations from Transformers) [6] are pre-trained on large text corpora via masked language modelling (MLM), yielding general-purpose representations that transfer well to downstream tasks with minimal task-specific fine-tuning.

2.1.2 Bi-Encoder Architecture

A bi-encoder, also called a dual encoder, consists of two transformer encoders, one for queries and one for documents, whose outputs are compressed into fixed-size dense vectors [23, 14]. In practice the two towers typically share weights. Given a query q and a document d , each is encoded independently:

$$\mathbf{e}_q = \text{pool}(\text{Enc}(q)), \quad \mathbf{e}_d = \text{pool}(\text{Enc}(d)), \quad (2.1)$$

where $\text{pool}(\cdot)$ is a pooling operation applied to the sequence of hidden states. Mean pooling, which averages all token hidden states, is standard for retrieval models and was established by Sentence-BERT (SBERT) [23]. Relevance is scored as the cosine similarity between the query and document embeddings:

$$\text{sim}(\mathbf{e}_q, \mathbf{e}_d) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\| \|\mathbf{e}_d\|}. \quad (2.2)$$

The architectural separation of query and document encoding is the critical property of bi-encoders for large-scale retrieval: document embeddings can be com-

puted offline and stored in an approximate nearest-neighbor (ANN) index, so that at query time only the query encoder runs and retrieval reduces to a single nearest-neighbor lookup [14]. This is in contrast to cross-encoders, which jointly encode the query and document and therefore cannot pre-compute document representations.

Exact exhaustive search, computing similarity against every document vector, is $\mathcal{O}(N \cdot d)$ per query, where N is the number of documents and d is the embedding dimension. At ten million documents with 768-dimensional embeddings this requires roughly 15 billion floating-point operations per query, making sub-millisecond latency unachievable on a single machine. Approximate nearest-neighbor (ANN) indices such as Hierarchical Navigable Small World (HNSW) [20] and Inverted File Index (IVF) [12] address this by organizing vectors so that only a small fraction of the corpus is visited per query, recovering the necessary speed at a small, controlled loss in recall. Specialized disk-resident indices such as DiskANN [28] narrow this memory requirement further through SSD-optimized graph traversal, but incur latency overheads of roughly three orders of magnitude over DRAM, making in-memory indexing the standard for interactive workloads. Even with ANN indexing, the full set of document embeddings must reside in memory for typical deployments; this memory requirement is the bottleneck this thesis addresses through embedding compression.

2.1.3 BGE-base-en-v1.5

This thesis evaluates compression methods across three backbone architectures to test whether compression trade-offs are model-specific or generalize. The primary backbone is BGE-base-en-v1.5 [34]. BGE (BAAI General Embedding) is a family of English bi-encoders released by the Beijing Academy of Artificial Intelligence (BAAI). The *base* variant uses a 12-layer BERT-like architecture with a hidden size of 768, producing 768-dimensional float32 embeddings via mean pooling. BGE-base-en-v1.5 was selected as the primary backbone because it achieves strong performance on the Massive Text Embedding Benchmark (MTEB) [21] at a moderate parameter count (109M parameters), and because its 768-dimensional output makes the memory costs and compression trade-offs studied in this thesis directly relevant to production deployments.

2.1.4 RoBERTa-base

RoBERTa (Robustly Optimized BERT Pretraining Approach) [18] is a replication study of BERT that identifies and corrects several under-training choices in the original recipe. Key differences from BERT include: removal of the Next Sentence Prediction (NSP) objective, dynamic masking (each training sequence receives a freshly sampled mask at each epoch rather than a static one), larger batch sizes, and substantially more training data (160 GB versus BERT’s 16 GB). These changes together yield stronger representations than BERT on most GLUE (a suite of natural language understanding tasks spanning sentiment,

inference, and similarity [32]) and SQuAD (a reading comprehension benchmark requiring span extraction from Wikipedia passages [22]) benchmarks while using the identical 12-layer, 768-dimensional transformer architecture (125M parameters). RoBERTa-base is included as a second backbone in this thesis to test whether the compression trade-offs observed on BGE, which is itself initialized from a retrieval-tuned BERT, also hold for a pre-training-optimized general-purpose encoder.

2.1.5 MPNet-base

MPNet (Masked and Permuted Pre-training) [26] addresses a dependency gap in masked language modeling: when BERT predicts a masked token it treats other masked positions as absent, so the model never explicitly learns dependencies among masked tokens. MPNet resolves this by combining masked language modeling with permuted language modeling (PLM, from XLNet): the model predicts all tokens autoregressively in a random permutation order while attending to the full original sequence, so masked-token dependencies are captured without the position-discrepancy problem of pure PLM. The result is a 12-layer, 768-dimensional encoder (110M parameters) that outperforms both BERT and RoBERTa on several sentence-level benchmarks. MPNet-base is included as a third backbone to provide a pre-training objective that is architecturally distinct from both BERT-style MLM and the retrieval-oriented BGE, further stress-testing the generality of the compression findings.

The three backbones span a range of pre-training quality for retrieval. BGE arrives already fine-tuned for retrieval and serves as the strongest starting point. MPNet produces richer sentence-level representations than RoBERTa owing to its masked-and-permuted objective, which captures dependencies between masked tokens that MLM-only models miss. RoBERTa, trained purely on token-level MLM, produces the weakest initial retrieval representations of the three. This ordering, $\text{BGE} > \text{MPNet} > \text{RoBERTa}$, is confirmed by the NanoBEIR zero-shot baseline results reported in Chapter 4. The central question this spread is designed to answer is whether compression trade-offs are stable across backbones of different starting quality, or whether a model that begins with weaker retrieval representations degrades differently under the same compression budget.

2.2 Contrastive Learning for Dense Retrieval

2.2.1 The InfoNCE Objective

Contrastive learning trains a model to embed semantically related items close together and unrelated items far apart [30]. The Information Noise-Contrastive Estimation loss (InfoNCE) formalizes this as a N -way classification task: given a query q paired with a positive document p^+ and $N - 1$ negative documents $\{p_i^-\}$, the model must identify the one correct document among all candidates. The model is trained to assign the highest similarity score to the positive pair:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(\mathbf{e}_q, \mathbf{e}_{p^+})/\tau)}{\exp(\text{sim}(\mathbf{e}_q, \mathbf{e}_{p^+})/\tau) + \sum_{i=1}^{N-1} \exp(\text{sim}(\mathbf{e}_q, \mathbf{e}_{p_i^-})/\tau)}, \quad (2.3)$$

where $\tau > 0$ is a temperature hyperparameter that controls how peaked the softmax distribution is: smaller τ sharpens the distribution toward the highest-scoring document, while larger τ flattens it across all candidates.

2.2.2 Multiple Negatives Ranking Loss

Multiple Negatives Ranking Loss (MNRL) applies InfoNCE in the sentence-transformers training framework [23]. In MNRL, negatives are obtained for free from the other examples in the same training batch: for a batch of N query-positive pairs $\{(q_i, p_i^+)\}_{i=1}^N$, the positive document of every other pair in the batch serves as a negative for query q_i . The loss for the full batch is:

$$\mathcal{L}_{\text{MNRL}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\text{sim}(\mathbf{e}_{q_i}, \mathbf{e}_{p_i^+})/\tau)}{\sum_{j=1}^N \exp(\text{sim}(\mathbf{e}_{q_i}, \mathbf{e}_{p_j^+})/\tau)}. \quad (2.4)$$

In-batch negatives are effective because large batch sizes expose each query to many diverse negatives simultaneously, and because batch-mates are often topically related to the query, providing challenging but informative negatives without any explicit negative mining [14]. When training data includes explicit triplets (i.e., mined hard negatives), MNRL extends naturally by appending them to the denominator alongside the in-batch negatives.

2.3 Embedding Compression Methods

Chapter 1 introduced a taxonomy of embedding compression methods along two axes: *when* compression is applied (post-hoc vs. training-time) and

how memory is reduced (precision reduction, dimensionality reduction, or learning to hash). This section provides technical depth for each method.

2.3.1 Scalar Quantization

Scalar quantization maps each float32 dimension to a lower-precision integer. For INT8 quantization, a calibration pass over a representative corpus sample computes per-dimension bounds $[\min_j, \max_j]$, where j indexes embedding dimensions. Each value x in dimension j is then quantized as:

$$x_q = \left\lfloor \frac{x - \min_j}{\delta_j} \right\rfloor - 128, \quad \delta_j = \frac{\max_j - \min_j}{255}, \quad (2.5)$$

where $\lfloor \cdot \rfloor$ denotes rounding to the nearest integer and the -128 shift maps the unsigned range $[0, 255]$ to the signed 8-bit range $[-128, 127]$. This *asymmetric* scheme uses the full 256 levels across the observed range of each dimension, reducing storage from 4 bytes to 1 byte per dimension ($4\times$ compression) [7].

The same calibration bounds are applied to both corpus and query embeddings, ensuring that corresponding dimensions are mapped to a consistent integer scale.

2.3.2 Binary Quantization

Binary quantization replaces each float32 dimension with a single bit by applying the sign function:

$$b_i = \text{sign}(x_i) \in \{-1, +1\}, \quad (2.6)$$

where x_i is the i -th dimension of the float32 embedding and b_i is the corresponding bit, yielding $32\times$ compression over float32. Similarity between binary query vector $\mathbf{b}_q$ and document vector $\mathbf{b}_d$ is computed via Hamming distance $d_H(\mathbf{b}_q, \mathbf{b}_d) = \sum_i \mathbf{1}[b_{q,i} \neq b_{d,i}]$, where $\mathbf{1}[\cdot]$ is the indicator function, which reduces to a bitwise XOR followed by a popcount instruction on modern hardware, enabling extremely fast similarity computation [8]. Applied post-hoc to a float32-trained model, binary quantization is aggressive and typically incurs a non-trivial drop in retrieval quality relative to INT8.

2.3.3 Product Quantization

Product quantization (PQ) [11] achieves high compression by dividing each d -dimensional embedding into M equal sub-vectors and representing each sub-vector by a single integer index rather than raw floats. For each of the M sub-spaces, a codebook of K centroids $\{\mathbf{c}_k^{(m)}\}$ is fit by k -means on the corpus embeddings. Each corpus sub-vector $\mathbf{e}^{(m)} \in \mathbb{R}^{d/M}$ is then replaced by the index of its nearest centroid:

$$\hat{k}^{(m)} = \arg \min_k \left\| \mathbf{e}^{(m)} - \mathbf{c}_k^{(m)} \right\|. \quad (2.7)$$

A document is stored as the sequence of M indices $(\hat{k}^{(1)}, \dots, \hat{k}^{(M)})$, each requiring $\log_2 K$ bits. For $M = 64$ sub-vectors and $K = 256$ centroids, a 768-dimensional float32 embedding shrinks from 3,072 bytes to 64 bytes, a $48\times$ compression ratio.

At retrieval time, corpus embeddings are decoded from their PQ codes back to approximate float32 representations using the learned centroids, while query

embeddings are kept at full float32 precision. Scoring then proceeds as standard cosine similarity between the reconstructed corpus and exact query vectors. Compressing only the document side while keeping the query exact (asymmetric reconstruction) recovers much of the quality lost by compression [12].

2.3.4 TurboQuant

TurboQuant [37] is a *post-hoc, precision-reduction* method: it is calibrated on corpus embeddings after the embedding model has run, with no retraining of the model itself. It claims near-optimal rate-distortion performance: for a given number of bits, it loses less retrieval-relevant information than scalar or product quantization methods can achieve. This is possible because TurboQuant fits a global non-linear quantizer that minimises reconstruction error across the full embedding dimension jointly, rather than partitioning dimensions into independent sub-codebooks. Originally developed for online quantization of key-value (KV) caches in large language model inference, its structure applies directly to offline retrieval index compression. At retrieval time, stored codes are decoded back to approximate float32 vectors, and scoring proceeds with standard cosine similarity.

2.3.5 Straight-Through Estimator Binarization

Post-hoc binary quantization may be substantially improved by incorporating the sign function into the model architecture during fine-tuning, a hypothesis this thesis evaluates empirically. The fundamental obstacle is that the

sign function is non-differentiable at zero and has zero derivative everywhere else, making standard backpropagation impossible. The Straight-Through Estimator (STE) [3] resolves this by using the sign function in the forward pass and passing gradients through it unchanged in the backward pass, as if the operation were the identity function:

$$\text{forward: } \hat{\mathbf{e}} = \text{sign}(\mathbf{e}), \quad \text{backward: } \frac{\partial \mathcal{L}}{\partial \mathbf{e}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{e}}}, \quad (2.8)$$

where $\mathbf{e}$ is the pre-binarization embedding and $\hat{\mathbf{e}}$ is its binarized counterpart. The estimator is biased (the passed gradient is not the true gradient of the sign function), yet **(author?)** report that it surprisingly outperforms more mathematically rigorous unbiased alternatives in practice. Training with the STE encourages the model to produce representations whose pre-binarization values are already close to ± 1 , so that the quantization step introduces minimal information loss at inference time. This approach is referred to in this thesis as Binarization-Aware Training (BAT) with the STE variant.

2.3.6 Annealed Tanh Binarization

The STE’s gradient approximation is coarse: gradients are propagated regardless of the magnitude of the pre-activation, even when values are near zero and the sign function is therefore poorly approximated by the identity. Annealed Tanh binarization replaces the STE with a smooth surrogate that progressively

sharpens toward the sign function over the course of training [4]:

$$\hat{\mathbf{e}} = \tanh(\beta \cdot \mathbf{e}), \quad (2.9)$$

where β is annealed according to the schedule $\beta_t = \sqrt{\gamma \cdot t + 1}$, with t the global optimizer step and γ a growth rate hyperparameter [35]. At the start of training ($t = 0$), $\beta = 1$ and $\tanh(\mathbf{e}) \approx \mathbf{e}$ for small activations, leaving the model in a nearly unconstrained float32 regime. As training progresses, β grows and $\tanh(\beta \cdot \mathbf{e})$ sharpens toward $\text{sign}(\mathbf{e})$, gradually tightening the binarization constraint. At inference time the hard sign function is applied, recovering a standard binary representation. The annealing schedule avoids the abrupt gradient discontinuity of the STE and allows the model to adapt its representations before the constraint becomes binding.

2.3.7 Matryoshka Representation Learning

Matryoshka Representation Learning (MRL) [15] trains a model to produce embeddings in which every leading m -dimensional prefix $\mathbf{e}_{1:m}$ is independently useful for retrieval. The core insight is that if the model is trained to solve the retrieval task at multiple granularities simultaneously, the first few dimensions will capture the most discriminative information, and later dimensions contribute increasingly fine-grained detail, analogous to the nested structure of Russian matryoshka dolls.

MRL augments the standard MNRL objective with losses computed at a fixed set of nested dimensionalities $\mathcal{M} = \{m_1, m_2, \dots, m_K\}$ where $m_1 < m_2 <$

$\dots < m_K = d$. The multi-granularity loss is:

$$\mathcal{L}_{\text{MRL}} = \sum_{m \in \mathcal{M}} c_m \cdot \mathcal{L}_{\text{MNRL}}(\mathbf{e}_{1:m}), \quad (2.10)$$

where c_m are positive scalar weights (set to uniform in this thesis) and $\mathcal{L}_{\text{MNRL}}(\mathbf{e}_{1:m})$ is the MNRL loss computed using only the first m dimensions. This thesis uses $\mathcal{M} = \{32, 64, 128, 256, 512\}$ for all MRL experiments.

At inference time, MRL allows the embedding to be truncated to any $m \in \mathcal{M}$ dimensions without additional fine-tuning, directly enabling post-hoc dimensionality reduction. This makes MRL embeddings qualitatively different from standard embeddings: truncating a standard embedding arbitrarily destroys retrieval quality, whereas an MRL-trained embedding is designed to remain useful at each supported granularity. The memory saving is proportional to the truncation ratio; using only the first 64 dimensions of a 512-dimensional MRL embedding yields an $8\times$ reduction.

2.4 Information Retrieval Evaluation

2.4.1 Ranking Metrics

Retrieval systems are evaluated by how well they rank relevant documents above irrelevant ones. All metrics in this thesis assume binary relevance: each corpus document is either relevant or not relevant to a given query. For each query, the retrieval system produces a ranked list of documents; metrics are computed at cut-off k , meaning only the top- k documents in the ranked list are considered.

Mean Reciprocal Rank (MRR@ k) [31] is the primary metric used in this thesis. It measures the position of the first relevant document in the ranked list:

$$\text{MRR@}k = \frac{1}{\text{rank}_1}, \quad \text{where } \text{rank}_1 = \min\{i \leq k : \text{doc}_i \in \mathcal{R}\}, \quad (2.11)$$

and $\text{MRR@}k = 0$ if no relevant document appears in the top k . MRR is particularly sensitive to the position of the single best hit and is well suited to retrieval tasks where there is typically one correct answer passage per query, as is the norm across most NanoBEIR subsets.

Normalized Discounted Cumulative Gain (NDCG@ k) [10] measures ranking quality by assigning higher credit to relevant documents appearing earlier in the list. The Discounted Cumulative Gain at position k is:

$$\text{DCG@}k = \sum_{i=1}^k \frac{\mathbf{1}[\text{doc}_i \in \mathcal{R}]}{\log_2(i+1)}, \quad (2.12)$$

where $\mathcal{R}$ is the set of relevant documents and $\mathbf{1}[\cdot]$ is the indicator function. $\text{NDCG@}k$ normalizes $\text{DCG@}k$ by the ideal DCG ($\text{IDCG@}k$), the score achieved by a perfect ranking that places all relevant documents first:

$$\text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}, \quad \text{IDCG@}k = \sum_{i=1}^{\min(|\mathcal{R}|, k)} \frac{1}{\log_2(i+1)}. \quad (2.13)$$

$\text{NDCG@}k$ lies in $[0, 1]$, with 1 indicating a perfect ranking.

Recall@ k measures the fraction of all relevant documents that are retrieved in the top k results:

$$\text{Recall@}k = \frac{|\{i \leq k : \text{doc}_i \in \mathcal{R}\}|}{|\mathcal{R}|}. \quad (2.14)$$

Recall@ k is bounded above by $\min(k, |\mathcal{R}|)/|\mathcal{R}|$ and equals 1 only when all relevant documents are retrieved within the top k positions.

Accuracy@ k (also called hit rate) is a binary metric indicating whether at least one relevant document appears in the top k results:

$$\text{Accuracy@}k = \mathbf{1}[\exists i \leq k : \text{doc}_i \in \mathcal{R}]. \quad (2.15)$$

All four metrics are computed at $k \in \{1, 3, 5, 10\}$, averaged over all queries in a NanoBEIR subset, and macro-averaged across the 13 subsets.

2.4.2 Statistical Significance Testing

Retrieval benchmarks consist of a fixed set of queries, making it possible to treat model comparisons as paired tests: each query yields one score per model, and the question is whether one model systematically outperforms another across the full query set.

Shapiro–Wilk normality test [25]. Before choosing which significance test to interpret, the Shapiro–Wilk test checks whether the differences $\{d_i\}$ follow a normal distribution:

- H_0 : the differences are normally distributed.

- H_1 : the differences are not normally distributed.

If H_0 is rejected, the normality assumption of the paired t -test is violated and the Wilcoxon test is the more appropriate choice.

Paired t -test [27]. A parametric test that assumes the differences are normally distributed and tests whether their mean is zero:

- H_0 : $\mu_d = 0$ (the two models have equal mean score).
- H_1 : $\mu_d \neq 0$ (one model has a higher mean score).

Wilcoxon signed-rank test [33]. A non-parametric alternative that makes no normality assumption. For each query, the difference d_i is ranked by magnitude; the test checks whether the positive-signed ranks (queries where model A wins) consistently outweigh the negative-signed ranks (queries where model B wins):

- H_0 : the median difference is zero (neither model is better).
- H_1 : the median difference is not zero.

Cohen’s d [5] measures the practical magnitude of the difference between two models. It is computed as the mean of the per-query differences divided by their standard deviation:

$$d = \frac{\bar{d}}{s_d}, \quad \bar{d} = \frac{1}{n} \sum_{i=1}^n d_i, \quad s_d = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (d_i - \bar{d})^2}. \quad (2.16)$$

Cohen’s d quantifies the practical magnitude of the difference: values of $|d| \approx 0.2$, 0.5, and 0.8 are conventionally interpreted as small, medium, and large effects

respectively. A statistically significant result with a small Cohen’s d indicates a reliable but practically negligible difference.

2.5 Related Work

Early evidence that discrete embeddings can match float32 retrieval quality came from Yamada et al. [35], whose Binary Passage Retriever (BPR) integrated learning-to-hash into the dense passage retrieval framework, achieving a $32\times$ memory reduction from 65 GB to 2 GB with zero accuracy loss on Natural Questions and TriviaQA. BPR uses a two-stage pipeline: binary Hamming search over the full corpus for candidate generation, followed by float32 inner product reranking over the shortlist. Training combines a margin ranking loss on the binary codes with MNRL on the float32 embeddings jointly. At production scale, Gan et al. [8] deployed binary retrieval at Tencent using recurrent residual binarization with STE for gradient flow, reporting 30–50% index cost reductions across web search, video retrieval, and copyright detection. The Hugging Face embedding quantization survey [24] established that even post-hoc binary and INT8 quantization, applied without retraining, retains strong MTEB quality when paired with asymmetric scoring. These results collectively establish that compressed embeddings are viable for large-scale retrieval. Each study evaluates a different base model on a different benchmark, however, so cross-method comparisons are unreliable: the effect of compression is conflated with differences in pre-training, fine-tuning, and data curation.

The key technical challenge in training-time compression is that quantization functions are non-differentiable. Two families of solutions have been established. The STE [3] passes gradients through the sign function unchanged. Courbariaux et al. [?] established it as the standard approach for binarizing neural network weights and activations; both BPR [35] and Gan et al. [8] rely on it for retrieval binarization. The alternative is the continuation approach of Cao et al. [4], which replaces the sign function with tanh activations whose temperature is progressively annealed during training until each tanh approximates a step function. BPR adapts this schedule to passage retrieval alongside STE. The annealed tanh experiments in this thesis build on the same lineage, applying temperature annealing within the sentence-transformers framework with MNRL as the sole training objective and no float32 reranking stage. STE and annealed tanh have each been used for retrieval binarization in prior work, but have never been compared directly under identical conditions; this thesis isolates that comparison. Beyond these binary approaches, Shah et al. [?] propose a Decoupled STE that uses separate temperatures for the forward and backward passes, reducing the bias inherent in the standard estimator. For multi-bit quantization, Yang et al. [?] represent a k -level quantizer as a sum of $k-1$ shifted sigmoid functions with learnable biases, each sharpened by an annealed temperature. Schrödter et al. [?] apply a similar sigmoid staircase to input feature compression, achieving 5–16 $\times$ compression with modest accuracy loss. Neither approach has been applied to retrieval embeddings; both are directions for future work.

Kusupati et al. [15] introduced MRL, training a single embedding so that any prefix of dimensions forms a useful representation. MRL now appears prominently on the MTEB leaderboard [21] and offers an attractive operating point when memory and latency budgets vary by deployment context. Sequential Matryoshka Embedding Compression (SMEC) [?] identifies three limitations of standard MRL: gradient fluctuation from jointly optimizing all dimension scales simultaneously, information degradation from blind truncation, and weak contrastive learning from easy in-batch negatives. Addressing these with sequential dimension training, adaptive dimension selection, and cross-batch memory, SMEC improves BEIR NDCG@10 by 1.9 points at 128 dimensions over MRL baselines on LLM-scale models. Quantization-Aware Matryoshka Adaptation (QAMA) [?], the closest prior work to this thesis, combines MRL with multi-level quantization-aware training. QAMA uses learnable thresholds initialized from percentile statistics and updated by exponential moving average, and applies a hybrid precision scheme that allocates higher bits to early MRL dimensions (2-bit for the first 25%) and progressively lower bits to later ones (0.5-bit for the final 25%), reaching an average of 1.625 bits per dimension while retaining 95–98% of full-precision NDCG@10. QAMA evaluates on ModernBERT and MiniLM, leaving open whether these findings generalize across architectures with different retrieval starting quality.

Controlled Retrieval Evaluation of Compression Techniques (CoRECT) [?] provides the most rigorous comparison of post-hoc methods to date, evaluating eight compression techniques across four embedding models on a benchmark

spanning 10K to 100M passages. Its findings are directly relevant to this thesis: quantization outperforms dimensionality truncation at equal compression ratios, MRL training matters significantly for truncation quality, and combining quantization with moderate truncation achieves the best recall for three of the four models tested. CoRECT does not fix the base model across compression types, however, and leaves training-time methods entirely out of scope. No existing study therefore enables a direct comparison between post-hoc and training-time compression under controlled conditions. Every prior study also evaluates a single backbone, leaving open whether compression trade-offs are model-specific or generalize across architectures with different retrieval starting quality. Stacked combinations involving training-time methods, such as MRL jointly trained with binarization, remain unstudied. This thesis addresses all three gaps: all compression variants share a single float32 baseline fine-tuned on the same corpus and are evaluated on NanoBEIR [38, 29] across BGE-base-en-v1.5, RoBERTa-base, and MPNet-base.

Chapter 3. Datasets

This chapter describes the two categories of data used in this thesis: the multi-source training corpora used to fine-tune the base embedding model, and the evaluation benchmark used to measure retrieval performance. These two categories are kept strictly separate; no training example appears in any evaluation set. Training data is drawn from publicly available question answering, paraphrase, and natural language inference corpora, while evaluation is performed exclusively on the NanoBEIR benchmark.

3.1 Training Data

The embedding model is fine-tuned on a mixture of seven publicly available text retrieval and semantic similarity datasets sourced from the Hugging Face Hub. Drawing from multiple domains and pair types reduces the risk of overfitting to a single retrieval pattern and improves the generalization of learned representations. Each dataset contributes (query, positive passage) pairs or (query, positive passage, negative passage) triplets that are consumed by the Multiple Negatives Ranking Loss during training.

3.1.1 Dataset Sources

The seven training datasets span five broad retrieval and semantic similarity tasks: passage retrieval, title-to-abstract prediction, duplicate question detection, natural language inference, and open-domain question answering. Table 3.1 summarizes each dataset, its pair type, raw size, assigned sampling weight, and the effective number of training examples drawn from it per training epoch under the weighted sampling scheme described in Section 3.1.2.

Table 3.1: Training dataset summary. Effective rows reflect the contribution of each dataset to the training mixture after weighted batch sampling. Total rows across all sources sum to 58.73M; effective rows sum to approximately 4.19M.

S.No.	Dataset	Pair Type	Total Rows	Weight	Effective Rows
1	msmacro_triplet	Query to Passage (triplet)	13.64M	0.15	~1.94M
2	s2orc_title_abstract	Title to Abstract (pair)	41.77M	0.01	~0.40M
3	quora_dup_triplet	Duplicate question (triplet)	2.79M	0.15	~0.40M
4	nli_for_simcse_triplet	Entailment/Contradiction (triplet)	0.27M	2	~0.52M
5	natural_questions	Question to Answer (pair)	0.10M	4	~0.38M
6	trivia_qa_triplet	Question to Answer (triplet)	0.06M	4	~0.23M
7	hotpotqa	Multi-hop Q to A (triplet)	0.08M	4	~0.32M
Total			58.73M		~4.19M

MS MARCO Triplets (*msmacro_triplet*) is derived from the Microsoft Machine Reading COmprehension dataset [2], the largest source in the mixture with 13.64 million (query, passage, negative passage) triplets. Queries are real user-issued Bing search queries and passages are extracted from web documents.

A low sampling weight of 0.15 is applied to prevent this large corpus from dominating gradient updates relative to the smaller, higher-quality datasets.

Semantic Scholar Open Research Corpus (*s2orc_title_abstract*) provides 41.77 million (title, abstract) pairs drawn from scientific publications across a wide range of academic disciplines [19]. This source trains the model to associate document titles with their corresponding abstracts, which serves as a proxy for document-level retrieval. The very low weight of 0.01 reflects the structural regularity of title-abstract pairs (titles are informationally dense summaries rather than natural-language queries) and prevents this very large scientific-domain corpus from skewing the mixture.

Quora Duplicate Triplets (*quora_dup_triplet*) contains 2.79 million (question, duplicate question, non-duplicate question) triplets drawn from the Quora Question Pairs dataset. This source trains the model to embed semantically equivalent questions in close proximity in representation space, which benefits retrieval tasks where queries may appear in paraphrased form.

NLI for SimCSE Triplets (*nli_for_simcse_triplet*) adapts natural language inference corpora following the SimCSE training recipe [9], yielding 0.27 million (premise, entailment hypothesis, contradiction hypothesis) triplets. Entailment pairs serve as high-quality positives and contradiction pairs serve as hard negatives. Despite its small size, this source receives the second-highest weight of 2, reflecting the quality of its hard negatives and the demonstrated effectiveness of NLI-derived pairs for contrastive representation learning.

Natural Questions (*natural_questions*) contains 0.10 million (question, answer passage) pairs extracted from Google’s Natural Questions dataset [16], in which questions are real user queries submitted to the Google search engine and answers are Wikipedia passages identified by human annotators. A high weight of 4 is assigned to compensate for the small size of this dataset relative to the mixture total, and because open-domain question-answering pairs closely match the target retrieval distribution of most benchmark tasks.

TriviaQA Triplets (*trivia_qa_triplet*) provides 0.06 million (trivia question, supporting passage, negative passage) triplets from the TriviaQA reading comprehension dataset [13]. Questions require broad factual knowledge and are paired with supporting evidence passages from Wikipedia and web documents. This source also receives a weight of 4 to counteract its small absolute size.

HotpotQA (*hotpotqa*) contributes 0.08 million multi-hop (question, supporting passage, negative passage) triplets [36]. HotpotQA questions require reasoning across multiple documents to answer, representing a more challenging retrieval scenario than single-hop question answering. A weight of 4 is applied, consistent with the other small question-answering sources.

3.1.2 Weighted Batch Sampling

Rather than concatenating all seven datasets and sampling uniformly, the training pipeline employs weighted batch sampling, where each dataset is assigned a scalar weight that controls the expected number of examples drawn from it per training step. Let w_i denote the weight of dataset i and N_i its total number

of examples. The effective contribution of dataset i to the training mixture per epoch is proportional to $w_i \cdot N_i$, scaled so that the contributions across all datasets sum to approximately 4.19 million effective examples (see Table 3.1).

This scheme serves two purposes. First, it prevents the two largest but more structurally constrained sources (MS MARCO and S2ORC, which together account for 94 percent of raw examples) from overwhelming the gradient signal with a single retrieval pattern. Second, it allows smaller domain-specific question-answering datasets (Natural Questions, TriviaQA, and HotpotQA) to contribute meaningfully to training despite their limited raw size. The resulting effective training mixture of approximately 4.19 million examples per epoch is substantially smaller than the 58.73 million raw examples, which reduces wall-clock training time while preserving the diversity of the training signal across retrieval tasks and domains.

3.2 Evaluation Benchmark

All experiments in this thesis are evaluated on the NanoBEIR benchmark. The full BEIR benchmark, from which NanoBEIR is derived, was considered but not used due to the prohibitive cost of encoding corpora containing millions of passages for every compression variant under study.

3.2.1 NanoBEIR

NanoBEIR [38] is a lightweight subset of the Benchmarking Information Retrieval (BEIR) benchmark [29], constructed by subsampling each BEIR dataset

to 50 evaluation queries and a small accompanying corpus. It is distributed on the Hugging Face Hub as part of the *sentence-transformers* ecosystem and is designed to enable rapid model comparisons without the computational cost of encoding the full BEIR corpora, making it well suited for iterative development and ablation studies.

This thesis uses all 13 available NanoBEIR subsets, summarized in Table 3.2. The subsets span a diverse set of retrieval tasks, including fact verification, question answering, entity retrieval, citation prediction, biomedical search, and argument retrieval, and cover domains ranging from general web content to financial documents to scientific literature. Each subset provides 50 queries (49 in the case of NanoTouche2020) and a corpus ranging from 2,260 to 6,100 documents, for a total of 649 queries and 57,390 corpus documents across all 13 subsets.

The diversity of NanoBEIR tasks is particularly important for evaluating compression methods. A compression technique that degrades performance on some tasks while improving or maintaining it on others would produce a misleading average score if evaluated on a homogeneous benchmark. By spanning 13 qualitatively different retrieval scenarios, NanoBEIR provides a more robust signal for assessing whether a compression method is broadly applicable.

Table 3.2: NanoBEIR benchmark subsets used for evaluation. All subsets contain 50 queries except NanoTouche2020, which contains 49. Corpus sizes range from 2,260 to 6,100 documents. The total across all subsets is 649 queries and 57,390 corpus documents.

S.No.	Subset	Queries	Corpus	Task (Query Type to Document Type)
1	NanoArguAna	50	3,690	Counter-argument retrieval (Claim to Argument)
2	NanoClimateFEVER	50	3,460	Fact-checking (Claim to Document)
3	NanoDBPedia	50	6,100	Entity retrieval (Query to Document)
4	NanoFEVER	50	5,050	Fact-checking (Claim to Document)
5	NanoFiQA2018	50	4,650	Financial question answering (Question to Document)
6	NanoHotpotQA	50	5,140	Multi-hop question answering (Question to Document)
7	NanoMSMARCO	50	5,090	Web search / general QA (Question to Passage)
8	NanoNFCorpus	50	3,000	Biomedical information retrieval (Query to Document)
9	NanoNQ	50	5,090	Open-domain question answering (Question to Document)
10	NanoQuoraRetrieval	50	5,100	Duplicate detection (Question to Question)
11	NanoSCIDOCs	50	2,260	Citation prediction (Title to Document)
12	NanoSciFact	50	2,970	Scientific fact-checking (Claim to Document)
13	NanoTouche2020	49	5,790	Conversational argument retrieval (Question to Argument)
Total		649	57,390	

Chapter 4. Methodology

This chapter describes how the experiments in this thesis were designed and executed. Section 4.1 presents the overarching experimental framework. Section 4.2 describes the shared training infrastructure and hyperparameters. Section 4.3 details the float32 baseline fine-tuning, which serves as the starting point for all post-hoc compression methods. Section 4.4 describes each training-time compression variant. Section 4.5 describes how post-hoc compression methods are calibrated and applied. Section 4.6 defines the stacked combinations evaluated. Section 4.7 describes the evaluation pipeline, metrics, memory budget calculation, and significance testing procedure.

4.1 Experimental Design

The central design principle of this thesis is controlled comparison: every compression method, whether post-hoc or training-time, is applied to the same set of backbone models fine-tuned under identical conditions, and evaluated on the same benchmark under the same pipeline. This eliminates the confounds that make cross-method comparisons unreliable in prior work (Section 2.5).

The experimental workflow has two parallel tracks, both rooted in the same pre-trained backbone weights. In the post-hoc track, each backbone is

fine-tuned without any compression modification, producing a float32 checkpoint; compression is then applied to the saved embeddings without touching model weights. In the training-time track, each backbone is fine-tuned independently from the same pre-trained weights with a modified architecture or loss function; no float32 checkpoint is used as initialization. All resulting embedding variants are evaluated on the NanoBEIR benchmark under the unified pipeline described in Section 4.7.

Table 4.1 summarises all six training variants and the post-hoc transforms available for each.

Table 4.1: Summary of training variants. Each variant is a complete fine-tuning run from a pre-trained backbone. Post-hoc transforms are applied after training with no weight modification.

Variant	Architecture change	Loss function	Post-hoc options
Float32 baseline	None	MNRL	INT8, Binary, PQ, TurboQuant
STE binarization-aware	+ BinarizationLayer (STE)	MNRL	Binary
MRL	None	MatryoshkaLoss (MNRL)	Truncation; Truncation + INT8/Binary
MRL + STE	+ BinarizationLayer (STE)	MatryoshkaLoss (MNRL)	Truncation; Truncation + Binary
Annealed tanh binarization-aware	+ AnnealedTanhLayer	MNRL	Binary
MRL + Annealed tanh	+ AnnealedTanhLayer	MatryoshkaLoss (MNRL)	Truncation; Truncation + Binary

4.2 Training Infrastructure and Hyperparameters

4.2.1 Backbone Models

All training variants are run independently on three backbone architectures: BGE-base-en-v1.5, RoBERTa-base, and MPNet-base (described in Section 2.1). No weights are shared across backbones.

4.2.2 Training Setup

Training is implemented with the *sentence-transformers* library [23] using the HuggingFace Trainer API. Multi-GPU training is performed via PyTorch Distributed Data Parallel (DDP) launched with `torchrun` across four GPUs. Gradients are synchronized across all ranks; checkpoint saving and mid-training evaluation are performed only on rank 0.

All models are trained in `bfloat16` precision where hardware support is available, falling back to `float16` otherwise. The `NO_DUPLICATES` batch sampler is used within each dataset to prevent the same (query, document) pair from appearing more than once per batch, preserving the diversity of in-batch negatives required by the MNRL objective.

4.2.3 Hyperparameters

Hyperparameters are shared across all training variants and backbone experiments; none are tuned per-variant. Table 4.2 summarises the complete configuration.

Table 4.2: Training hyperparameters shared across all experiments and backbone models. Effective batch size accounts for per-device batch, number of GPUs, and gradient accumulation.

Hyperparameter	Value
Optimizer	AdamW
Learning rate	2×10^{-5}
LR scheduler	Linear decay with warmup
Warmup ratio	0.10
Weight decay	0.10
Per-device batch size	16
Gradient accumulation steps	8
Effective batch size	512 (16 $\times$ 4 GPUs $\times$ 8 steps)
Training epochs	1
Max sequence length	512 tokens
Precision	bfloat16 (float16 fallback)

4.3 Baseline Fine-Tuning

The float32 baseline is produced by fine-tuning each backbone as a standard `SentenceTransformer` with mean pooling (Section 2.1). No compression layer is added. The training objective is the Multiple Negatives Ranking Loss (MNRL, Section 2.2) computed with cosine similarity; in-batch negatives are drawn from all other positive passages in the same batch. Training applies MNRL to batches drawn from all datasets; the `WeightedBatchSampler` controls how frequently each dataset contributes batches during an epoch (Chapter 3).

At the end of training the complete `SentenceTransformer` checkpoint (transformer weights, pooling configuration, and tokenizer) is saved to disk. This checkpoint serves as the common starting point for all post-hoc compression methods. Training-time compression experiments do not use this checkpoint; they are fine-tuned independently from the same pre-trained backbone weights.

4.4 Training-Time Compression Experiments

Each training-time experiment fine-tunes the pre-trained backbone directly, not the float32 baseline checkpoint, modifying either the model architecture, the loss function, or both. All other settings, including training infrastructure, hyperparameters, and training data, remain identical to the baseline.

4.4.1 STE Binarization-Aware Training

A `BinarizationLayer` is appended as an additional module after the mean-pooling layer in the `SentenceTransformer` module sequence. In the forward pass the layer applies `sign(·)` to the pooled float32 embedding, mapping each dimension to $\{-1, +1\}$. In the backward pass the layer acts as the identity (the Straight-Through Estimator, Section 2.3), passing gradients to the encoder unchanged. The loss function is unchanged from the baseline: MNRL with cosine similarity. Training encourages the encoder to produce pre-binarization activations already close to ± 1 , so that the discrete sign step introduces minimal information loss at inference time.

At inference, `sign(·)` is applied and the resulting bit-vector is packed into `uint8` for storage.

4.4.2 MRL Fine-Tuning

No architectural change is made; the model is identical to the baseline. The MNRL loss is wrapped by `MatryoshkaLoss`, which evaluates them at each of six nested granularities $\{32, 64, 128, 256, 512, 768\}$. The five truncation granularities $\{32, 64, 128, 256, 512\}$ are the supported post-hoc truncation points; the full-dimension loss ($d = 768$) is included to ensure the complete representation is not degraded during training. All six losses are weighted equally ($c_m = 1$ for all m , Equation 2.10).

At inference the MRL checkpoint is identical to the baseline float32 checkpoint; truncation to any supported granularity is applied post-hoc by the evaluation pipeline (Section 5.5).

4.4.3 Joint MRL and STE Binarization-Aware Training

This variant combines the architectural change of Section 4.4.1 with the loss modification of Section 4.4.2. The `BinarizationLayer` (STE) is inserted after mean pooling, and the loss is `MatryoshkaLoss` wrapping MNRL at all six granularities. The combination trains representations that are both nested and binarization-aware, enabling post-hoc truncation followed by binary packing.

4.4.4 Annealed Tanh Binarization-Aware Training

An `AnnealedTanhBinarizationLayer` replaces the STE layer after mean pooling, applying the annealed tanh surrogate described in Section 2.3. The annealing rate is $\gamma = 0.1$, following [35]; the empirical validation of this choice across $\gamma \in \{0.05, 0.1, 0.2\}$ is reported in Table 12. At approximately $t = 8,000$ steps (one epoch over the full training mixture), $\beta \approx 28$, at which point $\tanh(\beta_t \cdot \mathbf{e}) \approx \text{sign}(\mathbf{e})$ for typical activation magnitudes.

A `BetaAnnealCallback` increments t after every optimizer step and logs β to Weights & Biases. At inference the layer switches to the hard `sign(·)` function. The loss function is MNRL with cosine similarity applied to the soft-binarized embeddings during training.

4.4.5 Joint MRL and Annealed Tanh Binarization-Aware Training

This variant combines the `AnnealedTanhBinarizationLayer` of Section 4.4.4 with the `MatryoshkaLoss` of Section 4.4.2. Architecture and loss are modified simultaneously, training representations that are both nested and binarization-aware via the smooth annealed surrogate.

4.5 Post-Hoc Compression

All post-hoc methods are applied after the float32 checkpoint has been trained; they do not modify the model weights. Raw float32 embeddings are encoded once and cached (Section 4.7); the compression transform is applied at evaluation load time.

4.5.1 INT8 Scalar Quantization

Following the procedure in Section 2.3 (Equation 2.5), calibration bounds are derived from the first 1,000 corpus embeddings after any dimensionality truncation. The same bounds are applied identically to corpus and query embeddings. Similarity is cosine similarity on the quantized representations.

4.5.2 Binary Quantization

Post-hoc binary quantization follows the procedure in Section 2.3. Bits are packed into `uint8` format using the *sentence-transformers* `quantize_embeddings`

utility. Similarity is Hamming similarity between packed binary query and corpus vectors.

4.5.3 Product Quantization

PQ is fitted on corpus embeddings using Faiss [7] following the procedure in Section 2.3. $M \in \{16, 24, 32, 48, 64, 96, 128, 192\}$ sub-spaces are evaluated with centroid bit-widths of $n_{\text{bits}} \in \{4, 8\}$.

4.5.4 TurboQuant

TurboQuant [37] is calibrated on the corpus embeddings via the `TurboQuantMSE` fitter (Section 2.3). Bit-depths of $\{1, 2, 3, 4, 8\}$ bits per dimension are evaluated.

4.6 Stacked Combinations

MRL-trained checkpoints support simultaneous compression along two dimensions: dimensionality reduction via post-hoc truncation, followed by any precision-reducing method. Specifically, each MRL checkpoint is truncated to each of the five supported granularities $\{32, 64, 128, 256, 512\}$, and then binary quantization, PQ, or TurboQuant is applied to the truncated embedding. This yields a grid of (backbone $\times$ truncation dimension $\times$ precision method) combinations evaluated in Chapter 5.

Each combination is registered as a separate `ModelSpec` in `eval_v2/config/models.py`. All variants that share the same checkpoint reuse a single cached float32 embed-

ding, so encoding is performed only once per checkpoint regardless of how many compression variants are derived from it.

4.7 Evaluation Framework

4.7.1 Embedding Cache and Transform Pipeline

Raw float32 embeddings are computed once per checkpoint and stored under a content-addressed cache. The cache key is the first 16 hex characters of the SHA-256 hash of the model path, ensuring distinct checkpoints produce distinct cache entries while variants sharing a checkpoint reuse a single cached copy. For each NanoBEIR subset, corpus and query embeddings are stored as compressed NumPy arrays (`.npz`) together with a JSON sidecar recording the model path, embedding count, and dimension. A cache miss triggers a multi-GPU encoding pass.

At evaluation time, the transform pipeline is applied to the cached float32 embeddings in the following fixed order:

1. Dimensionality truncation (if `truncate_dim` is set).
2. Binary quantization and uint8 packing (if `similarity = hamming`).
3. INT8 scalar quantization (if `quant_bits = 8`).
4. PQ fitting and asymmetric reconstruction (if `pq_M` is set); fitted on corpus.
5. TurboQuant quantization and dequantization (if `tq_bits` is set); fitted on corpus.

Steps 4 and 5 are mutually exclusive; only one precision-reducing transform is active per model variant.

4.7.2 Metrics

The four retrieval metrics used in this thesis, $\text{NDCG}@k$, $\text{MRR}@k$, $\text{Accuracy}@k$, and $\text{Recall}@k$, are defined in Section 2.4. All are computed at $k \in \{1, 3, 5, 10\}$, averaged over the 50 queries per NanoBEIR subset, then macro-averaged across the 13 subsets. MRR is the primary reported metric throughout Chapter 5.

4.7.3 Memory Budget and Compression Ratio

Embedding memory is computed as:

$$\text{memory (bytes)} = n \times d_{\text{stored}} \times \frac{b}{8}, \quad (4.1)$$

where n is the number of embeddings, d_{stored} is the stored dimension (number of PQ sub-spaces for PQ variants), and b is the number of bits per stored element. For float32, $b = 32$; for INT8, $b = 8$; for binary packed embeddings, $b = 1$. Compression ratio is reported relative to the float32 baseline at the same embedding dimension.

4.7.4 Statistical Significance Testing

The tests used here are defined in Section 2.4.2. For each model pair, per-query scores are collected across all 13 NanoBEIR subsets and pooled into a single paired sample (649 query pairs per comparison). The per-query difference

is $d_i = s_A(q_i) - s_B(q_i)$ for each query q_i , where $s(\cdot)$ denotes the retrieval score of the respective model. The Shapiro–Wilk test is first applied to this vector of differences to assess normality. If normality is not rejected, the paired t -test is used; otherwise the Wilcoxon signed-rank test is used. Cohen’s d is reported as an effect size in either case. Results in Chapter 5 are flagged as statistically significant at $\alpha = 0.05$.

Experiments and Results

5.1 Evaluation Setup

5.1.0.0.1 Benchmark. All models are evaluated on NanoBEIR [38], a down-sampled variant of the Benchmarking Information Retrieval (BEIR) benchmark [29] in which each corpus is capped at a few thousand passages to enable rapid comparison of large numbers of model variants. Thirteen task subsets are included, spanning open-domain question answering, fact verification, entity search, argument retrieval, and scientific information retrieval [Table 5.1]. Each subset provides 50 test queries (49 for NanoTouche2020) and a corpus of 1,866–6,045 passages, yielding 649 total queries and 56,352 total corpus passages across all subsets.

Table 5.1: NanoBEIR subsets used for evaluation.

[Table 5.1 placeholder: subset name — domain/task type — # queries — corpus size]

5.1.0.0.2 Encoder backbones. Three 768-dimensional sentence encoders serve as backbones: (i) BAAI/bge-base-en-v1.5 (BGE), a retrieval-specialised model; (ii) roberta-base (RoBERTa), a general-purpose masked language model; and (iii) all-mpnet-base-v2 (MPNet), a general-purpose sentence encoder. Each backbone

is evaluated in two regimes: the publicly released pre-trained checkpoint and a retrieval fine-tuned version trained with Multiple Negatives Ranking Loss (MNRL) over eight mixed retrieval datasets, as described in Chapter 4. Because all three backbones produce 768-dimensional float32 embeddings in their default configuration, they share a common reference point for cross-method and cross-backbone comparison.

5.1.0.0.3 Compression methods. Compression configurations are drawn from three families, each introduced in Chapter 4: *post-hoc precision reduction* (INT8, binary quantisation, Product Quantisation (PQ), and TurboQuant (TQ)); *training-time binarisation* (the Straight-Through Estimator binarisation layer (BAT) and annealed tanh binarisation (atanh) at three annealing rates $\gamma \in \{0.05, 0.10, 0.20\}$); and *training-time dimensionality reduction* via Matryoshka Representation Learning (MRL), which produces truncated float32 embeddings at five sub-dimensions (32, 64, 128, 256, 512). Selected MRL checkpoints are further combined with binary quantisation, TQ, and PQ to yield stacked compression.

5.1.0.0.4 Compression ratio. The compression ratio r is defined relative to the float32 baseline: a 768-dimensional float32 embedding occupies $768 \times 4 = 3,072$ bytes. For scalar precision reduction to b bits per component, $r = 32/b$ (e.g. INT8: $4\times$; binary: $32\times$). For MRL truncation to d' dimensions, $r = 768/d'$. For PQ with M subspaces and n -bit codes per subspace, $r = (768 \times 32)/(M \times n)$. Combined methods multiply the per-axis ratios; for example, MRL-256 followed by binary quantisation gives $(768/256) \times 32 = 96\times$.

5.1.0.0.5 Evaluation metrics. The primary metric is Mean Reciprocal Rank at depth 10 (MRR@10), macro-averaged equally over all 13 subsets. MRR@10 reflects the rank of the first relevant document in the top-10 retrieved results and is well-suited to short-list retrieval settings. Secondary metrics, Normalised Discounted Cumulative Gain at 10 (NDCG@10) and Recall at k for $k \in \{1, 3, 5, 10\}$, are computed for all configurations but serve only as supplementary analysis.

5.1.0.0.6 Similarity functions. Float32 and INT8 embeddings are scored by cosine similarity. Binary embeddings (one bit per original dimension, packed as uint8) are scored by Hamming distance, which is equivalent to inner-product similarity over $\{-1, +1\}$ codes and admits fast bitwise XOR-popcount computation. PQ and TQ retrieval use asymmetric distance estimation: the query embedding is decoded to float32 before scoring while corpus embeddings remain in their compressed form.

5.1.0.0.7 Statistical significance. Pairwise significance tests are conducted for every model pair on each NanoBEIR subset using a paired permutation test applied to per-query MRR scores at significance level $\alpha = 0.05$. Aggregated pairwise significance patterns are visualised as a heat map in [Figure 5.X].

5.2 Float32 Baselines

Table 5.2 reports mean MRR@10 macro-averaged across all 13 NanoBEIR subsets for each backbone in its pre-trained and fine-tuned float32 configuration.

These results serve as the upper-bound reference for all compression experiments in Sections 5.3–5.6.

Table 5.2: Mean MRR@10 (macro-averaged over 13 NanoBEIR subsets) for pre-trained and fine-tuned float32 embeddings. Δ is the absolute change due to fine-tuning.

[Table 5.2 placeholder: Configuration — BGE — RoBERTa — MPNet — Δ
(BGE) — Δ (RoBERTa) — Δ (MPNet)]

5.2.0.0.1 Effect of fine-tuning. The three backbones exhibit qualitatively different responses to retrieval fine-tuning. RoBERTa-base and MPNet-base enter with near-chance MRR@10 scores of 0.097 and 0.156 respectively, consistent with their pre-training objectives (masked language modelling and general-purpose sentence similarity) providing no direct signal for asymmetric passage retrieval. Fine-tuning with Multiple Negatives Ranking Loss (MNRL) over eight mixed retrieval datasets yields gains of 43.4 and 42.6 percentage points, lifting both models to competitive retrieval performance (0.530 and 0.582). This confirms that the contrastive fine-tuning procedure is effective at reshaping general-purpose representations into retrieval-oriented embedding spaces regardless of the pre-training objective.

BGE-base-en-v1.5 (BGE) behaves differently: fine-tuning produces a small but consistent *decrease* of 0.6 percentage points ($0.691 \rightarrow 0.685$). BGE has already undergone large-scale retrieval pre-training; further adaptation on the comparatively small mixed-domain corpus used here offers no additional benefit and may introduce minor distribution mismatch. This has a direct consequence for compression evaluation: for BGE, the fine-tuned float32 ceiling is slightly *below* the

pre-trained ceiling, so apparent recovery toward 0.691 under compression should be interpreted with care. Unless stated otherwise, all compression results in subsequent sections are reported relative to the fine-tuned float32 for each backbone.

5.2.0.0.2 Cross-backbone gap. Even after fine-tuning, a substantial absolute gap separates BGE from the two general-purpose backbones: BGE exceeds fine-tuned MPNet by 6.6 percentage points and fine-tuned RoBERTa by 15.5 percentage points. This gap reflects differences in backbone capacity and the depth of upstream retrieval pre-training rather than the fine-tuning procedure itself. The spread motivates the cross-backbone generalisability analysis in Section 5.7: whether compression methods preserve rank ordering across backbones or interact differently with each embedding geometry is examined there.

5.3 Post-Hoc Compression

This section evaluates compression applied after training to the fine-tuned float32 checkpoints, with no modification to model weights. Four methods are examined in increasing order of compression depth: INT8 scalar quantisation (Section 5.3.1), post-hoc binary quantisation (Section 5.3.2), Product Quantisation (Section 5.3.3), and TurboQuant (Section 5.3.4). Section 5.3.5 places all four methods on a common axis at 32× compression.

5.3.1 INT8 Scalar Quantisation

INT8 quantisation rounds each embedding component from float32 to a signed 8-bit integer, reducing per-vector storage from 3,072 bytes to 768 bytes ($4\times$ compression), with similarity still computed by cosine distance in float after dequantisation.

Table 5.3: Mean MRR@10 for INT8 quantisation ($4\times$) compared with the fine-tuned float32 baseline.

[Table 5.X placeholder: Configuration — BGE — RoBERTa — MPNet]

Across all three backbones, INT8 quantisation is essentially lossless (Table 5.3). BGE incurs a loss of 0.7 percentage points ($0.685 \rightarrow 0.678$) and MPNet 1.5 percentage points ($0.582 \rightarrow 0.567$), while RoBERTa shows a marginal *gain* of 0.2 percentage points ($0.530 \rightarrow 0.532$). The RoBERTa gain is within noise but consistent with INT8 rounding acting as a mild regulariser on embeddings that were not originally trained for retrieval. These results confirm that $4\times$ precision reduction via INT8 incurs negligible quality cost for all backbones, making it the recommended default for memory-constrained deployments where float32 precision is not required.

5.3.2 Binary Quantisation

Post-hoc binary quantisation thresholds each component at zero, producing one-bit-per-dimension embeddings packed as uint8 vectors (96 bytes per vector; $32\times$ compression). Retrieval is performed via Hamming distance.

Table 5.4: Mean MRR@10 for post-hoc binary quantisation ($32\times$) compared with the fine-tuned float32 baseline.

[Table 5.X placeholder: Configuration — BGE — RoBERTa — MPNet]

At $32\times$ compression, post-hoc binary quantisation inflicts meaningful quality degradation across all backbones (Table 5.4): BGE drops 3.9 percentage points ($0.685 \rightarrow 0.646$), MPNet drops 4.8 percentage points ($0.582 \rightarrow 0.534$), and RoBERTa drops 2.9 percentage points ($0.530 \rightarrow 0.501$). The degradation is expected: collapsing 32 bits to a single sign bit with no adaptation of the embedding geometry discards the magnitude information that encodes fine-grained relevance distinctions. Section 5.4 shows that training-time binarisation methods substantially recover this loss by learning representations whose sign more faithfully encodes retrieval signal.

5.3.3 Product Quantisation

Product Quantisation (PQ) partitions the embedding space into M disjoint subspaces and independently quantises each with an n -bit codebook learned from the corpus. Two configurations are evaluated, both at $32\times$ compression: $M=96$ subspaces with 8-bit codes (PQ- $m96-n8$) and $M=192$ subspaces with 4-bit codes (PQ- $m192-n4$). Retrieval uses asymmetric distance estimation, keeping only the query vector in float32 at query time.

Table 5.5: Mean MRR@10 for PQ configurations ($32\times$) compared with the fine-tuned float32 baseline and post-hoc binary.

[Table 5.X placeholder: Configuration — BGE — RoBERTa — MPNet]

PQ substantially outperforms post-hoc binary quantisation at the same $32\times$ ratio (Table 5.5). For BGE, PQ-*m96-n8* reaches 0.679, recovering over 80 % of the gap between post-hoc binary and the float32 baseline (3.3 percentage points recovered of 3.9). For MPNet the gain over post-hoc binary is 2.8 percentage points (0.534 $\rightarrow$ 0.562). RoBERTa benefits less: both configurations score within 0.001 of each other (0.514 and 0.519), suggesting the RoBERTa embedding geometry is less amenable to the Voronoi-cell structure assumed by PQ after fine-tuning.

Between the two PQ configurations, PQ-*m96-n8* is preferable for BGE and MPNet (0.679 vs 0.675 and 0.562 vs 0.549), while PQ-*m192-n4* has a marginal edge for RoBERTa (0.519 vs 0.514). The 8-bit-per-subspace variant benefits from finer codebook resolution at the cost of a larger number of centroids per subspace.

5.3.4 TurboQuant

TurboQuant (TQ) is a corpus-fitted quantiser that rotates embeddings into a whitened basis before applying uniform scalar quantisation at b bits per component. Three bit-widths are evaluated: 1-bit ($32\times$), 2-bit ($16\times$), and 4-bit ($8\times$).

[Figure 5.X placeholder: TQ quality–compression curve for all three backbones, with float32 baseline marked as a horizontal dashed line.]

Figure 5.1: Mean MRR@10 vs compression ratio for TurboQuant across bit-widths and backbones.

At 4-bit precision ($8\times$ compression), TQ is virtually lossless for all backbones (Figure 5.1). BGE scores 0.685, matching the fine-tuned float32 ceiling to within 0.03 percentage points; RoBERTa marginally exceeds its float32 baseline

(0.532 > 0.530); and MPNet scores 0.579 against a float32 ceiling of 0.582 (−0.3 percentage points). This near-lossless behaviour at $8\times$ makes TQ-4bit the best post-hoc method by quality-per-byte up to that compression level.

At 2-bit precision ($16\times$), TQ achieves 0.678, 0.530, and 0.574 for BGE, RoBERTa, and MPNet respectively. For BGE, TQ-2bit ($16\times$, 192 bytes) is Pareto-dominated by PQ-*m96-n8* ($32\times$, 96 bytes): PQ achieves higher MRR@10 (0.679 vs 0.678) at half the memory cost. This relationship does not hold for RoBERTa and MPNet, where TQ-2bit retains a quality advantage over PQ at any compression ratio (0.530 vs 0.514 and 0.574 vs 0.562).

At 1-bit precision ($32\times$), TQ (0.666, 0.519, 0.547) outperforms post-hoc binary quantisation (0.646, 0.501, 0.534) by 2.1, 1.7, and 1.4 percentage points for BGE, RoBERTa, and MPNet respectively. The whitening rotation redistributes variance more uniformly across dimensions before thresholding, preserving more retrieval signal per bit than a raw sign quantisation.

5.3.5 Cross-Method Comparison at $32\times$

[Figure 5.X placeholder: grouped bar chart of MRR@10 at $32\times$ for post-hoc binary, TQ-1bit, PQ-*m192-n4*, and PQ-*m96-n8*, across all three backbones.]

Figure 5.2: Mean MRR@10 at $32\times$ compression for all post-hoc methods across backbones. Float32 fine-tuned scores shown as dashed reference lines.

Table 5.6: Mean MRR@10 at $32\times$ compression. Float32 fine-tuned scores are shown for reference ($r = 1\times$).

[Table 5.X placeholder: Method — r — BGE — RoBERTa — MPNet]

Figure 5.2 and Table 5.6 place all post-hoc methods at a common compression budget. For BGE and MPNet, PQ-*m96-n8* is the strongest post-hoc method at 32 $\times$: it exceeds TQ-1bit by 1.2 and 1.4 percentage points, and post-hoc binary by 3.3 and 2.8 percentage points. For RoBERTa, PQ-*m192-n4* and TQ-1bit are essentially tied (0.519 vs 0.519), while PQ-*m96-n8* slightly underperforms TQ-1bit (0.514 vs 0.519), indicating that the optimal PQ configuration is backbone-dependent. Post-hoc binary quantisation is the weakest method at this budget across all three backbones.

Two broader observations emerge. First, learned quantisation (PQ) consistently outperforms scalar thresholding (post-hoc binary) at the same compression ratio, confirming that fitting the codebook to the corpus distribution is worth the offline compute cost. Second, TQ-1bit’s advantage over post-hoc binary (up to 2.1 percentage points for BGE) is smaller than PQ’s advantage over TQ-1bit (up to 1.4 percentage points for MPNet), suggesting that the whitening rotation alone recovers less information than structured codebook learning. Training-time methods, examined in Section 5.4, provide a complementary path to closing this remaining gap without relying on corpus-side fitting.

5.4 Training-Time Compression

Unlike post-hoc methods, training-time compression modifies the fine-tuning procedure itself so that the model learns representations optimised for the compressed form. Two families are evaluated: binarisation-aware training (Section 5.4.1), which targets the same 32 $\times$ compression ratio as post-hoc binary

quantisation, and Matryoshka Representation Learning (Section 5.4.2), which targets dimensionality reduction.

5.4.1 Binarisation-Aware Training: STE and Annealed Tanh

Two binarisation-aware training methods are evaluated. The Straight-Through Estimator binarisation layer (BAT) inserts a $\text{sign}(x)$ operation into the forward pass and passes gradients through unchanged; the embedding geometry is thus shaped by the retrieval loss throughout training. The annealed tanh method (atanh) instead uses $\tanh(\beta \cdot x)$ during training, where β grows from 1 to ≈ 28 over the course of fine-tuning, progressively tightening the approximation to $\text{sign}(x)$; three annealing rates $\gamma \in \{0.05, 0.10, 0.20\}$ are evaluated, with higher γ producing faster annealing. Both methods produce 768-dimensional binary embeddings at inference ($32\times$ compression, 96 bytes per vector).

Table 5.7: Mean MRR@10 at $32\times$ compression for binarisation-aware training methods, post-hoc binary, and the float32 baseline.

[Table 5.X placeholder: Method — BGE — RoBERTa — MPNet]

[Figure 5.X placeholder: grouped bar chart comparing post-hoc binary, BAT, and atanh- $\gamma 0.2$ at $32\times$ across all three backbones, with float32 dashed reference lines.]

Figure 5.3: Mean MRR@10 at $32\times$ compression for post-hoc binary, BAT, and annealed tanh ($\gamma=0.2$) across backbones.

Table 5.7 and Figure 5.3 show that binarisation-aware training consistently and substantially outperforms post-hoc binary quantisation at $32\times$ for all backbones. BAT gains 1.3, 6.4, and 4.8 percentage points over post-hoc binary for

BGE, RoBERTa, and MPNet respectively. The improvement is largest for the general-purpose backbones, confirming that these models benefit more from having their embedding geometry explicitly shaped for binary retrieval. For MPNet, BAT reaches 0.582, matching the fine-tuned float32 baseline to within 0.01 percentage points, a near-lossless result at $32\times$ compression. For RoBERTa, BAT reaches 0.565, exceeding the float32 baseline by 3.5 percentage points, consistent with the view that contrastive fine-tuning with a binarisation constraint can discover a more retrieval-effective geometry than unconstrained float32 training.

Annealed tanh at $\gamma=0.2$ follows a similar pattern but with a backbone-dependent trade-off relative to BAT. For RoBERTa, $\text{atanh}(\gamma=0.2)$ reaches 0.580, a further 1.5 percentage points above BAT and 5.0 percentage points above the float32 baseline, the strongest binary result for RoBERTa across all methods. The gradual annealing schedule appears to give the RoBERTa geometry more time to reorganise around binary boundaries before the binarisation constraint fully activates. For BGE and MPNet, however, BAT outperforms $\text{atanh}(\gamma=0.2)$ by 1.0 and 0.8 percentage points respectively (0.658 vs 0.648 and 0.582 vs 0.573), suggesting that the harder straight-through gradient signal is more effective for backbones that begin with a stronger retrieval prior. The results for $\gamma \in \{0.05, 0.10\}$ (Table 5.7) show a monotonic relationship between annealing rate and RoBERTa performance, with slower annealing consistently under-performing $\gamma=0.2$.

5.4.2 Matryoshka Representation Learning

Matryoshka Representation Learning (MRL) replaces the standard ranking loss with a weighted sum of losses computed at five nested sub-dimensions $d \in \{32, 64, 128, 256, 512\}$. The resulting model produces a single 768-dimensional float32 embedding whose leading d dimensions are independently informative, enabling truncation to any trained sub-dimension at deployment time without retraining.

[Figure 5.X placeholder: MRL truncation curve (MRR@10 vs d) for all three backbones, with the float32 768-dim baseline marked as a horizontal dashed line.]

Figure 5.4: Mean MRR@10 as a function of MRL truncation dimension d for all three backbones. The float32 768-dim baseline is shown as a dashed reference line for each backbone.

Table 5.8: Mean MRR@10 for MRL truncations at 512 and 256 dimensions, with compression ratio and float32 baseline shown for reference.

[Table 5.X placeholder: Truncation — r — BGE — RoBERTa — MPNet]

Figure 5.4 and Table 5.8 show that MRL achieves remarkably graceful quality degradation as d decreases. At 512 dimensions ($1.5\times$ compression), MRL matches or marginally exceeds the full 768-dimensional float32 baseline for BGE and RoBERTa: BGE reaches 0.687 against a float32 ceiling of 0.685 (+0.2 percentage points), and RoBERTa reaches 0.533 vs 0.530 (+0.3 percentage points). MPNet scores 0.579, within 0.3 percentage points of float32 (0.582). The slight gains for BGE and RoBERTa reflect MRL’s training incentive to concentrate retrieval signal into early dimensions: the 512-dimensional prefix may discard

low-variance tail dimensions that contribute noise rather than signal in the full 768-dimensional space.

At 256 dimensions ($3\times$ compression), quality remains within 1.0 percentage points of float32 for all backbones: BGE scores 0.675 (-1.0 pp), RoBERTa 0.526 (-0.4 pp), and MPNet 0.577 (-0.5 pp). The curve in Figure 5.4 shows that degradation accelerates as d falls below 128, where the information bottleneck becomes more severe. The primary value of MRL truncation, however, lies not in stand-alone compression but in enabling stacked combinations with binary quantisation, TQ, and PQ, which are examined in Section 5.5.

5.5 Stacked Combinations

Stacked combinations pair dimensional reduction (MRL truncation) with precision reduction to reach compression ratios beyond what either method achieves alone. Four pairings are evaluated: MRL+post-hoc binary and MRL+BAT at $48\times$, and MRL+TQ and MRL+PQ across a range of memory budgets.

Table 5.9: Mean MRR@10 for stacked combinations, with compression ratio and memory footprint per vector. MRL float32 baselines are included for reference.

[Table 5.X placeholder: Method — r — Bytes — BGE — RoBERTa — MPNet]

5.5.0.0.1 MRL+post-hoc binary versus MRL+BAT. Both MRL-512+post-hoc binary and MRL-512+BAT produce 512-bit binary vectors (64 bytes; $48\times$ compression), but differ in whether binarisation is applied post-hoc to the MRL float32 checkpoint or jointly trained from scratch. In the standalone setting (Sec-

tion 5.4.1), BAT substantially outperforms post-hoc binary for all backbones. The stacked results in Table 5.9 reveal a striking reversal: for BGE and MPNet, post-hoc binary applied to MRL-512 embeddings *outperforms* jointly trained MRL+BAT by 4.0 and 1.9 percentage points respectively (BGE: 0.633 vs 0.593; MPNet: 0.540 vs 0.521). For RoBERTa the advantage of BAT is preserved but sharply reduced to 1.6 percentage points (0.517 vs 0.501), compared with the 6.4 percentage point gap in the standalone setting.

This reversal suggests a significant optimisation conflict in joint MRL+BAT training. MRL’s loss requires the leading sub-dimensions to be informative as float32 representations across five nested budgets, while the straight-through binarisation gradient simultaneously pushes the geometry toward sign-separable boundaries. These constraints appear to work against each other for BGE and MPNet, whose retrieval-specialised pre-training already anchors the embedding space strongly. The two-stage approach (first train MRL for float32 truncation quality, then apply post-hoc binary) avoids this conflict by optimising each objective independently. RoBERTa is the exception: its weaker retrieval prior gives joint training more latitude to reorganise the embedding geometry, preserving the BAT advantage.

5.5.0.0.2 MRL+TQ versus MRL+PQ. MRL truncation compounds favourably with both TQ and PQ, enabling compression ratios in the $32\times$ – $96\times$ range while retaining meaningful retrieval quality. Table 5.9 and Figure 5.5 report results for the best-performing configurations at each memory budget.

[Figure 5.X placeholder: MRL+TQ and MRL+PQ quality–compression curves for all three backbones, with standalone TQ and PQ curves shown as dashed references.]

Figure 5.5: Mean MRR@10 vs compression ratio for MRL+TQ and MRL+PQ stacked combinations. Standalone TQ and PQ results are shown as dashed reference curves.

For BGE, MRL+PQ-*m64-n8* reaches 0.673 at $48\times$ (64 bytes), exceeding MRL-256+TQ-3bit (0.666 at $32\times$, 96 bytes) by 0.7 percentage points at *higher* compression. MRL+PQ therefore Pareto-dominates MRL+TQ on the BGE quality–compression frontier in this range: it achieves better quality with less memory. For RoBERTa and MPNet this relationship is reversed: MRL+TQ-3bit scores 0.523 and 0.576 against MRL+PQ-*m64-n8*’s 0.515 and 0.565, so TQ retains a quality advantage when more memory is available, consistent with the backbone-dependent behaviour observed in Section 5.3.5.

The MRL+TQ combination also yields the best results at extreme compression for BGE. Taking the best-performing MRL+TQ configuration at each memory budget, the quality envelope for BGE is: 0.373 at 8 bytes ($384\times$), 0.614 at 32 bytes ($96\times$), and 0.666 at 96 bytes ($32\times$). The 32-byte operating point is particularly notable: at $96\times$ compression, BGE retains 0.614 MRR@10, recovering substantial retrieval capability at a memory cost of just over 2% of the float32 baseline. The full stacked quality–compression curves are shown in Figure 5.5 and discussed further in the Pareto analysis of Section 5.6.

5.6 Pareto Analysis

A configuration is Pareto-optimal if no other configuration achieves both higher retrieval quality *and* higher compression ratio. The Pareto frontier therefore identifies the set of non-dominated operating points from which a practitioner should choose, given their memory budget. This section characterises the frontier for each backbone (Section 5.6.1), identifies the best available method at key compression budgets (Section 5.6.2), and analyses which compression families, post-hoc versus training-time, appear on the frontier and why (Section 5.6.3).

5.6.1 Pareto Frontiers

[Figure 5.X placeholder: Pareto scatter plots (MRR@10 vs compression ratio, log x -axis) for all three backbones, with frontier points connected by a step line and off-frontier points shown as hollow markers. Training-time methods in a distinct colour.]

Figure 5.6: Quality–compression Pareto frontiers for BGE, RoBERTa, and MPNet. Compression ratio is shown on a log scale. Filled markers indicate Pareto-optimal configurations; hollow markers are dominated.

The BGE Pareto frontier (Figure 5.6) spans four orders of magnitude in compression ratio and is detailed in Table 5.10. Its composition is as follows: pre-trained BGE ($1\times$, 0.691) $\rightarrow$ MRL-512 ($1.5\times$, 0.687) $\rightarrow$ TQ-4bit ($8\times$, 0.685) $\rightarrow$ TQ-3bit ($10.7\times$, 0.680) $\rightarrow$ PQ- $m96-n8$ ($32\times$, 0.679) $\rightarrow$ MRL-256+PQ- $m64-n8$ ($48\times$, 0.673) $\rightarrow$ PQ- $m48-n8$ ($64\times$, 0.663) $\rightarrow$ MRL-256+PQ- $m32-n8$ ($96\times$, 0.637) $\rightarrow \dots \rightarrow$ MRL-32+PQ- $m4-n4$ ($1536\times$, 0.201).

Table 5.10: BGE Pareto-optimal configurations, ordered by compression ratio. Method family is indicated for each point.

[Table 5.X placeholder: Method — Family — r — Bytes — MRR@10]

Three structural features of the BGE frontier deserve attention. First, the pre-trained checkpoint (0.691) appears at $1\times$ because fine-tuning slightly *reduces* BGE quality (Section 5.2); the fine-tuned float32 is thus dominated by its own pre-trained counterpart at the same memory cost. Second, TQ occupies the moderate-compression range ($8\times$ – $10.7\times$) with remarkably small quality loss: the step from $1.5\times$ to $8\times$ costs only 0.2 percentage points. Third, PQ takes over from $32\times$ onwards: at this ratio, PQ-*m96-n8* (0.679) outperforms every other method including TQ-1bit (0.666) and BAT (0.658), both of which are off the frontier at $32\times$. Beyond $32\times$, stacked MRL+PQ combinations extend the frontier monotonically, with each step compounding both compression dimensions.

The RoBERTa and MPNet frontiers differ from BGE in one important respect: training-time binarisation methods appear as Pareto-optimal points at $32\times$. For RoBERTa, annealed tanh ($\gamma=0.2$) achieves 0.580 at $32\times$, exceeding the best post-hoc method at that ratio (PQ-*m192-n4*: 0.519) by 6.1 percentage points; for MPNet, BAT achieves 0.582, exceeding the best post-hoc method (PQ-*m96-n8*: 0.562) by 2.0 percentage points. These gaps are large enough that no post-hoc configuration at any compression ratio dominates the training-time methods for these backbones, making them unambiguously Pareto-optimal at $32\times$.

Table 5.11: Best-performing method and MRR@10 at four representative compression budgets for each backbone. Ties broken by compression ratio (higher preferred).

[Table 5.X placeholder: Budget — Best method (BGE) — MRR@10 — Best method (RoBERTa) — MRR@10 — Best method (MPNet) — MRR@10]

5.6.2 Best Method at Each Compression Budget

Table 5.11 summarises the recommended configuration at four operating points.

At $4\times$, INT8 quantisation is the clear choice for all backbones: BGE retains 0.678, RoBERTa 0.532, and MPNet 0.567, each within 1.5 percentage points of float32 with no corpus fitting or retraining required.

At $8\times$, TQ-4bit achieves near-lossless compression across all three backbones (BGE: 0.685, RoBERTa: 0.532, MPNet: 0.579), matching or approaching the float32 baseline at one-eighth the memory. This operating point offers the best quality-per-byte of any method evaluated once the overhead of float32 is considered excessive.

At $32\times$, the optimal method is backbone-dependent. For BGE, PQ- $m96-n8$ (0.679) leads all methods, including training-time alternatives. For RoBERTa and MPNet, training-time binarisation dominates: annealed tanh ($\gamma=0.2$) is best for RoBERTa (0.580) and BAT is best for MPNet (0.582), both substantially ahead of the best post-hoc methods at this ratio.

At $48\times$, MRL-256+PQ- $m64-n8$ is the strongest configuration for BGE (0.673), extending the frontier via stacked compression with less than 2 percentage points of additional quality loss relative to $32\times$. For RoBERTa and MPNet,

no stacked combination at $48\times$ outperforms the training-time methods at $32\times$; practitioners targeting these backbones at extreme compression are better served by accepting the $32\times$ training-time result than by pushing further with stacked combinations.

5.6.3 Training-Time versus Post-Hoc Methods on the Frontier

The Pareto analysis reveals a clear division of labour between the two compression paradigms. For BGE, the frontier is composed entirely of post-hoc methods (TQ, PQ) and stacked combinations built on the MRL float32 checkpoint; no training-time binarisation method (BAT or annealed tanh) appears as a Pareto-optimal point. At $32\times$, the canonical comparison ratio for binary retrieval, PQ-*m96-n8* (0.679) dominates BAT (0.658) and annealed tanh (0.648), meaning that fitting a product codebook to the corpus is more effective than shaping the embedding geometry during training for a backbone that already carries a strong retrieval prior.

For RoBERTa and MPNet, the picture is reversed at $32\times$: training-time methods are firmly on the frontier and post-hoc methods cannot reach them. The gap of 6.1 percentage points between annealed tanh and the best post-hoc method for RoBERTa is larger than the total quality loss from post-hoc binary alone (2.9 pp), underscoring how profoundly binarisation-aware training reshapes the embedding geometry when the backbone lacks a retrieval prior.

This backbone-conditioned split has a practical implication: the choice between training-time and post-hoc compression should be informed by the re-

trieval specialisation of the backbone. For retrieval-specialised models such as BGE, post-hoc corpus-fitted quantisation achieves superior results without requiring any retraining. For general-purpose models such as RoBERTa and MPNet, investment in binarisation-aware fine-tuning is necessary to reach the Pareto frontier at high compression ratios. No single strategy dominates across all backbones, reinforcing the need for the controlled cross-backbone comparison conducted in this thesis.

5.7 Cross-Backbone Generalisability

The three backbones span a wide range of retrieval specialisation, from the retrieval-pretrained BGE to the general-purpose RoBERTa and MPNet. This section examines whether compression methods behave consistently across that range: whether rank orderings are stable, whether the magnitude of compression-induced quality loss is backbone-invariant, and whether conclusions drawn on one backbone transfer to others.

[Figure 5.X placeholder: quality-retention heat map (MRR@10 relative to each backbone’s float32 baseline) for all methods and all three backbones, grouped by compression ratio.]

Figure 5.7: Quality retention (compressed MRR@10 / float32 MRR@10) for all evaluated methods across backbones. Values above 1.0 indicate the compressed model exceeds its float32 baseline.

5.7.0.0.1 Low-compression regime (up to 8×). INT8 (4×) and TQ-4bit (8×) are the most backbone-agnostic methods evaluated. INT8 incurs losses of

0.7, -0.2 (gain), and 1.5 percentage points for BGE, RoBERTa, and MPNet respectively; TQ-4bit incurs losses of 0.03, -0.2 (gain), and 0.3 percentage points. In both cases the direction of effect is consistent (near-zero) and the quality ordering of backbones is perfectly preserved: $\text{BGE} > \text{MPNet} > \text{RoBERTa}$ throughout. MRL truncation to 512 dimensions ($1.5\times$) similarly shows uniform behaviour: all three backbones match or marginally exceed their float32 baselines (BGE $+0.2$ pp, RoBERTa $+0.3$ pp, MPNet -0.3 pp). At 256 dimensions ($3\times$), losses remain small and consistent (BGE -1.0 pp, RoBERTa -0.4 pp, MPNet -0.5 pp). These results indicate that for compression up to $8\times$, and for MRL truncation up to $3\times$, the choice of backbone does not materially affect the choice of compression method: all methods are near-lossless and all rank orderings are stable.

5.7.0.0.2 High-compression regime ($32\times$). At $32\times$, backbone identity becomes the dominant factor in method selection. Post-hoc binary quantisation degrades all backbones by 2.9–4.8 percentage points in absolute terms, and its rank ordering relative to float32 is stable (BGE retains 94 %, RoBERTa 94 %, MPNet 92 %). However, the relative effectiveness of other methods diverges sharply.

For post-hoc learned quantisation (PQ), BGE benefits most: PQ retains 99 % of its float32 quality, whereas RoBERTa and MPNet retain 97 % and 97 % respectively, a smaller but still meaningful advantage over raw binary. For training-time binarisation, the effect is strongly backbone-dependent in direction, not just magnitude: BAT reduces BGE quality by 2.7 percentage points below float32 but *raises* RoBERTa quality by 3.5 percentage points and leaves MPNet essentially unchanged (-0.01 pp). Annealed tanh ($\gamma=0.2$) amplifies this pattern further,

raising RoBERTa by 5.0 percentage points above float32 while reducing BGE by 3.7 percentage points. These opposing effects mean that a practitioner benchmarking on BGE would conclude that training-time binarisation underperforms post-hoc PQ, a conclusion that is the reverse of the truth for RoBERTa.

5.7.0.0.3 Rank-ordering stability. At low compression, method rankings are fully consistent across backbones. At $32\times$, they are not. The most consequential reversal is between training-time and post-hoc methods: for BGE, $PQ > BAT > \text{post-hoc binary}$; for RoBERTa and MPNet, $BAT \text{ and } \text{atanh} > PQ > \text{post-hoc binary}$. Within post-hoc methods, however, the ordering is more stable: PQ consistently outperforms TQ-1bit which consistently outperforms post-hoc binary across all backbones, with only minor variation in the margin. This suggests that post-hoc method rankings are transferable across backbones, while conclusions about training-time versus post-hoc compression are not.

5.7.0.0.4 Backbone-independent observations. Several findings hold uniformly across all three backbones and can be stated without qualification. First, INT8 and TQ-4bit are effectively lossless at $4\times$ and $8\times$ respectively. Second, post-hoc binary is always the weakest method at $32\times$, and learned quantisation (PQ or training-time) always outperforms it. Third, MRL truncation is the most backbone-consistent compression method at every truncation level, with quality losses that are small, monotonically increasing with compression ratio, and nearly uniform across backbones. Fourth, all stacked MRL+precision combinations de-

grade more gracefully for BGE than for RoBERTa and MPNet, reflecting the stronger retrieval geometry of the retrieval-pretrained backbone.

These observations together suggest that evaluation on a single retrieval-specialised backbone, as is common in prior work, is sufficient to characterise post-hoc methods but insufficient to characterise training-time binarisation, whose benefit is disproportionately concentrated on general-purpose backbones.

5.8 Discussion

5.8.0.0.1 Embedding geometry as the central explanatory variable.

The most consistent explanatory factor across results is the *retrieval geometry* of the backbone embedding space: the degree to which directions in the float32 space already encode retrieval relevance in a form amenable to compression. BGE, which has undergone large-scale retrieval pre-training, enters the compression experiments with a geometry that is already near-optimal for dense retrieval; the sign of each dimension carries substantial relevance signal, and corpus-fitted codebooks (PQ) can exploit this structure without retraining. RoBERTa and MPNet, whose float32 spaces are shaped by masked language modelling and general sentence similarity objectives, do not share this property. Their signs are not retrieval-informative until the model is explicitly trained to make them so.

This geometry hypothesis explains several otherwise puzzling results. It explains why binarisation-aware fine-tuning raises RoBERTa and MPNet *above* their float32 baselines at 32×: the binary constraint forces the model to discover a retrieval-oriented geometry that float32 training, without the constraint, never

finds. It explains why PQ, which adapts to the existing geometry without changing it, performs comparably to training-time binarisation for BGE but falls well short for RoBERTa and MPNet. And it explains why fine-tuning slightly *hurts* BGE: the backbone’s pre-trained geometry is already well-adapted for retrieval, and further adaptation on a smaller mixed-domain corpus provides no benefit and may introduce minor distributional noise.

5.8.0.0.2 Post-hoc corpus fitting as a substitute for geometry adaptation. A secondary theme is the value of corpus-fitted quantisation (PQ and TQ) as a practical alternative to training-time methods. Both PQ and TQ learn a non-linear mapping from the float32 embedding space to a compact code using only unlabelled corpus embeddings, with no label information or retraining. For BGE, this approach reaches the Pareto frontier at every compression ratio tested, demonstrating that when the underlying geometry is sound, corpus-side adaptation is sufficient. The Pareto dominance of PQ- $m96-n8$ over TQ-2bit for BGE (where PQ achieves higher quality at twice the compression ratio) illustrates that structured codebook learning extracts substantially more information from a good geometry than scalar rotation-and-threshold operations. The practical implication is that practitioners deploying retrieval-specialised backbones can avoid the cost and complexity of retraining by investing instead in corpus-calibrated quantisation.

5.8.0.0.3 Limits of joint optimisation in stacked compression. The MRL+BAT reversal (Section 5.5) reveals a non-obvious failure mode of joint

multi-objective training: when two compression objectives impose conflicting geometric constraints, joint optimisation can produce embeddings that are worse than either objective applied independently. MRL’s loss requires the leading d dimensions to be informative float32 representations across five nested truncation sizes; the straight-through binarisation gradient simultaneously pushes the geometry toward sign-separable boundaries. The conflict is resolved differently by different backbones: for RoBERTa, which has more latitude to reorganise its geometry, the joint training succeeds; for BGE and MPNet, it does not. A sequential strategy of training MRL for float32 truncation quality and then applying post-hoc binary quantization avoids the conflict entirely and consistently outperforms joint training for these backbones. This result suggests that the design of multi-axis compression pipelines should consider constraint compatibility explicitly rather than assuming that jointly trained objectives reinforce one another.

5.8.0.0.4 Limitations. Several limitations constrain the scope of conclusions that can be drawn from this study. First, NanoBEIR is a downsampled benchmark: corpora of 2,000–6,000 passages are smaller than production retrieval settings by one to two orders of magnitude, and PQ codebook quality in particular may differ at larger scale. Second, only three backbone architectures are evaluated; while they span a range of retrieval specialisation, the findings may not generalise to decoder-based or cross-encoder models, or to backbones with non-standard embedding dimensions. Third, PQ codebook storage is not counted in the per-vector memory budget, which understates the total memory cost of

PQ relative to simpler methods at small corpus scales. Fourth, the γ sweep for annealed tanh covers only three values ($\{0.05, 0.10, 0.20\}$); the optimal annealing schedule for each backbone may lie outside this range. Finally, all evaluations are on English-language datasets; whether the geometry hypothesis and the backbone-conditioned method rankings extend to multilingual or cross-lingual retrieval remains an open question.

References

- [1] Faizan Ahemad. Quantization aware matryoshka adaptation: Leveraging matryoshka learning, quantization, and bitwise operations for reduced storage and improved retrieval speed. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM)*, 2025.
- [2] Amazon Web Services. Amazon EC2 memory optimized instances (x2gd). <https://aws.amazon.com/ec2/instance-types/x2gd/>, 2024. Accessed: 2024.
- [3] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. MS MARCO: A human generated MACHINE Reading COMprehension dataset. In *Proceedings of the Workshop on Cognitive Computation: Integrating Neural and Symbolic Approaches (CoCo@NIPS)*, 2016.
- [4] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- [5] Zhangjie Cao, Mingsheng Long, Jianmin Wang, and Philip S. Yu. HashNet: Deep learning to hash by continuation. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 5608–5617, 2017.
- [6] Ilias Caspar and Michael Dinzinger. CoRECT: A framework for evaluating embedding compression techniques at scale. *arXiv preprint arXiv:2510.19340*, 2025.
- [7] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition, 1988.
- [8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186, 2019.

- [9] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *arXiv preprint arXiv:2401.08281*, 2024.
- [10] Yukang Gan, Yixiang Ding, Qifan Guo, Jingang Wang, and Wei Wu. Binary embedding-based retrieval at tencent. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 3966–3977, 2023.
- [11] Tianyu Gao, Xingcheng Yao, and Danqi Chen. SimCSE: Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6510–6526, 2021.
- [12] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.
- [13] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2010.
- [14] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.
- [15] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 1601–1611, 2017.
- [16] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781, 2020.
- [17] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, et al. Matryoshka representation learning. In *Advances in Neural Information Processing Systems*, volume 35, pages 30233–30249, 2022.

- [18] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019.
- [19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [20] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [21] Kyle Lo, Lucy Lu Wang, Mark Neumann, Rodney Kinney, and Daniel S. Weld. S2ORC: The semantic scholar open research corpus. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4969–4983, 2020.
- [22] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–837, 2020.
- [23] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2014–2037, 2023.
- [24] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, 2016.

- [25] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019.
- [26] Karsten Schrödter, Jan Stenkamp, Nina Herrmann, and Fabian Gieseke. Trainable bitwise soft quantization for input feature compression. In *Proceedings of the Third Conference on Parsimony and Learning (CPAL)*, 2026.
- [27] Rushi Shah, Mingyuan Yan, Michael Curtis Mozer, and Dianbo Liu. Improving discrete optimisation via decoupled straight-through estimator. *arXiv preprint arXiv:2410.13331*, 2024.
- [28] Aamir Shakir, Tom Aarsen, and Sean Lee. Embedding quantization. <https://huggingface.co/blog/embedding-quantization>, March 2024. Hugging Face Blog. Accessed: 2024.
- [29] Samuel Sanford Shapiro and Martin Bradbury Wilk. An analysis of variance test for normality (complete samples). *Biometrika*, 52(3–4):591–611, 1965.
- [30] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. MPNet: Masked and permuted pre-training for language understanding. In *Advances in Neural Information Processing Systems*, volume 33, pages 16857–16867, 2020.
- [31] Student. The probable error of a mean. *Biometrika*, 6(1):1–25, 1908.
- [32] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [33] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- [34] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

- [35] Ellen M. Voorhees. The TREC-8 question answering track report. In *Proceedings of the 8th Text REtrieval Conference (TREC-8)*, pages 77–82, 1999.
- [36] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the EMNLP 2018 Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355, 2018.
- [37] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.
- [38] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.
- [39] Ikuya Yamada, Akari Asai, and Hannaneh Hajishirzi. Efficient passage retrieval with hashing for open-domain question answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, pages 979–990, 2021.
- [40] Jiwei Yang, Xu Shen, Jun Xing, Xinmei Tian, Houqiang Li, Bing Deng, Jianqiang Huang, and Xian-Sheng Hua. Quantization networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7308–7316, 2019.
- [41] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018.
- [42] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate. In *The Thirteenth International Conference on Learning Representations*, 2026.
- [43] Zeta Alpha. NanoBEIR: A collection of smaller versions of BEIR datasets. <https://huggingface.co/collections/zeta-alpha-ai/>

nanobeir-66e1a0af21dfd93e620cd9f6, 2024. Hugging Face dataset collection.

- [44] Biao Zhang, Lixin Chen, Tong Liu, and Bo Zheng. SMEC: Rethinking matryoshka representation learning for retrieval embedding compression. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

Appendix A: Annealing Rate Ablation

A.1 Effect of γ on Annealed Tanh Binarization-Aware Training

Table 12 reports the mean MRR@10 across all 13 NanoBEIR subsets for each backbone model trained under three values of the annealing rate $\gamma \in \{0.05, 0.1, 0.2\}$. No single value of γ consistently outperforms the others across all backbones. BGE-base favours $\gamma = 0.2$ (wins on 8 of 13 subsets), while RoBERTa-base favours $\gamma = 0.1$ (7 of 13) and MPNet-base favours $\gamma = 0.05$ (6 of 13). The maximum MRR@10 spread within any backbone is 0.012, indicating that the results are not practically sensitive to this hyperparameter. We fix $\gamma = 0.1$ throughout as a principled default following [35].

Table 12: Mean MRR@10 across 13 NanoBEIR subsets for each backbone trained under three values of the annealing rate γ . Bold indicates the best value per row.

Backbone	$\gamma = 0.05$	$\gamma = 0.1$	$\gamma = 0.2$
BGE-base-en-v1.5	0.640	0.638	0.648
RoBERTa-base	0.576	0.590	0.580
MPNet-base	0.585	0.580	0.573